Java 基础面试题

大家好，我是小林。

Java 基础这块，在面试里的地位有点特殊——它不像 MySQL、Redis 那样每次都是核心考点，但如果你答得不好，会给面试官留下"基本功不扎实"的印象，这个代价其实挺大的。很多人用 Java 写了好几年代码，却说不清楚 JVM 和 JDK 的关系、int 和 Integer 的区别、为什么要重写 hashCode，这些问题看起来简单，但答起来很容易露馅。

这篇文章整理了 Java 基础面试中最常被问到的知识点，涵盖核心概念、数据类型、面向对象、关键字、反射、注解、异常、Java 8 新特性、IO 等内容，覆盖比较全面。内容偏向理解，不是让你背定义，而是帮你搞清楚"这个东西在 Java 里为什么是这样设计的"。

有几块在面试里出现频率特别高，建议重点花时间：

- **面向对象三大特性**：封装、继承、多态，尤其是多态的几种体现形式、重载和重写的区别，这类题看起来基础，但很多人答得很浅，容易被追问到哑口无言。
- **int 和 Integer**：为什么要有包装类、自动装箱拆箱的原理、Integer 的缓存机制（-128 到 127），这条线问得很频繁，而且很容易出坑。
- **String、StringBuffer、StringBuilder**：三者的区别和适用场景，以及 String 不可变性的原理，这是一道老题，但每次还是会问。
- **equals 和 hashCode**：为什么要配套重写、不重写会有什么问题，在 HashMap 和 HashSet 里的影响，这个很多人只知道结论，答不出背后的原因。
- **Java 8 新特性**：Lambda 表达式、Stream API、Optional，这些在实际开发里用得很多，面试里也经常问。
- **反射机制**：是什么、怎么用、实际应用在哪里（Spring 的 IOC 就是典型），这块理解清楚了对后面学框架原理很有帮助。

如果你是第一次系统准备 Java 基础，建议先把面向对象和数据类型搞清楚，再去看反射、注解这些稍微深一点的内容，整体理解起来会顺很多。

概念

说一下 Java 的特点

主要有以下的特点：

- **平台无关性**：Java 的“编写一次，运行无处不在”哲学是其最大的特点之一。Java 编译器将源代码编译成字节码（bytecode），该字节码可以在任何安装了 Java 虚拟机（JVM）的系统上运行。
- **面向对象**：Java 是一门严格的面向对象编程语言，几乎一切都是对象。面向对象编程（OOP）特性使得代码更易于维护和重用，包括类（class）、对象（object）、继承（inheritance）、多态（polymorphism）、抽象（abstraction）和封装（encapsulation）。
- **内存管理**：Java 有自己的垃圾回收机制，自动管理内存和回收不再使用的对象。这样，开发者不需要手动管理内存，从而减少内存泄漏和其他内存相关的问题。

Java 的优势和劣势是什么?

首先，Java 的优势，我记得跨平台应该是一个大点，因为 JVM 的存在，一次编写到处运行。然后面向对象，这个可能也是优势，不过现在很多语言都支持面向对象，但是 Java 的设计从一开始就是 OOP 的。还有强大的生态系统，比如 Spring 框架，Hibernate，各种库和工具，社区支持大，企业应用广泛。另外，内存管理方面，自动垃圾回收机制，减少了内存泄漏的问题，对开发者友好。还有多线程支持，内置的线程机制，方便并发编程。安全性方面，Java 有安全模型，比如沙箱机制，适合网络环境。还有稳定性，企业级应用长期使用，版本更新也比较注重向后兼容。

劣势的话，性能可能是一个，虽然 JVM 优化了很多，但相比 C++或者 Rust 这种原生编译语言，还是有一定开销。特别是启动时间，比如微服务场景下，可能不如 Go 之类的快。语法繁琐，比如样板代码多，之前没有 lambda 的时候更麻烦，现在有了但比起 Python 还是不够简洁。内存消耗，JVM 本身占内存，对于资源有限的环境可能不太友好。还有面向对象过于严格，有时候写简单程序反而麻烦，虽然 Java8 引入了函数式编程，但不如其他语言自然。还有开发效率，相比动态语言如 Python，Java 需要更多代码，编译过程也可能拖慢开发节奏。

Java 为什么是跨平台的?

Java 能支持跨平台，主要依赖于 JVM 关系比较大。

JVM 也是一个软件，不同的平台有不同的版本。我们编写的 Java 源码，编译后会生成一种 .class 文件，称为字节码文件。Java 虚拟机就是负责将字节码文件翻译成特定平台下的机器码然后运行。

也就是说，只要在不同平台上安装对应的 JVM，就可以运行字节码文件，运行我们编写的 Java 程序。

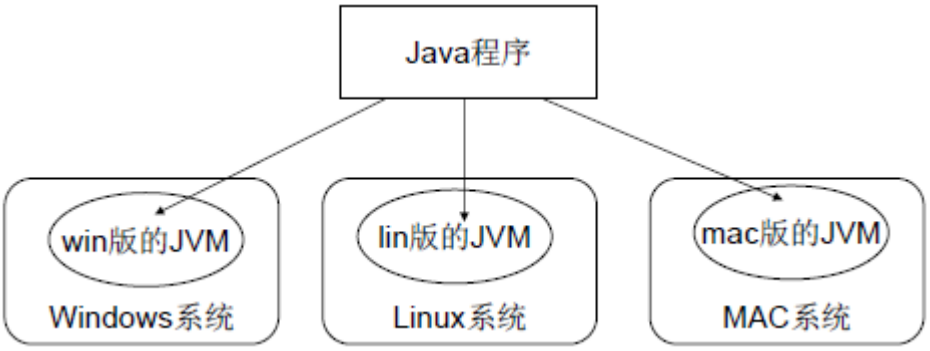
而这个过程中，我们编写的 Java 程序没有做任何改变，仅仅是通过 JVM 这一”中间层“，就能在不同平台上运行，真正实现了”一次编译，到处运行“的目的。

JVM 是一个”桥梁“，是一个”中间件“，是实现跨平台的关键，Java 代码首先被编译成字节码文件，再由 JVM 将字节码文件翻译成机器语言，从而达到运行 Java 程序的目的。

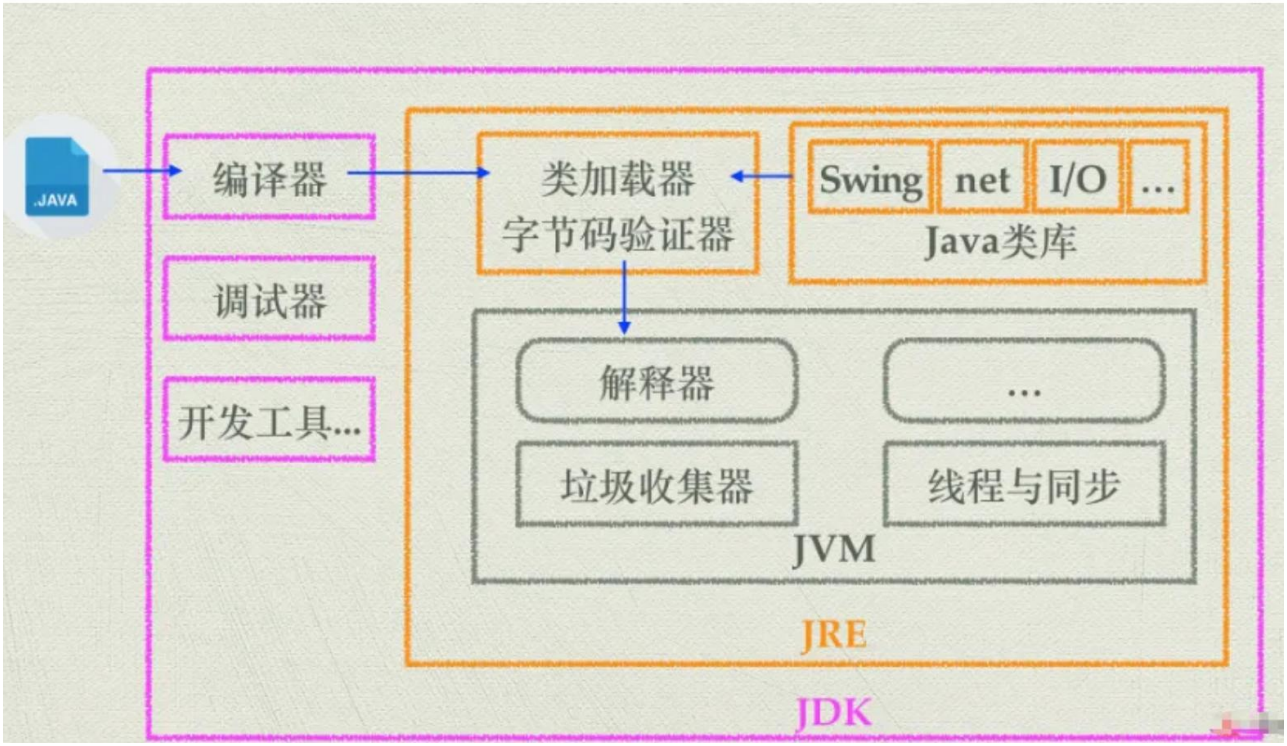
编译的结果不是生成机器码，而是生成字节码，字节码不能直接运行，必须通过 JVM 翻译成机器码才能运行。不同平台下编译生成的字节码是一样的，但是由 JVM 翻译成的机器码却不一样。

所以，运行 Java 程序必须有 JVM 的支持，因为编译的结果不是机器码，必须要经过 JVM 的再次翻译才能执行。即使你将 Java 程序打包成可执行文件（例如 .exe），仍然需要 JVM 的支持。

跨平台的是 Java 程序，不是 JVM。JVM 是用 C/C++开发的，是编译后的机器码，不能跨平台，不同平台下需要安装不同版本的 JVM。



JVM、JDK、JRE 三者关系?



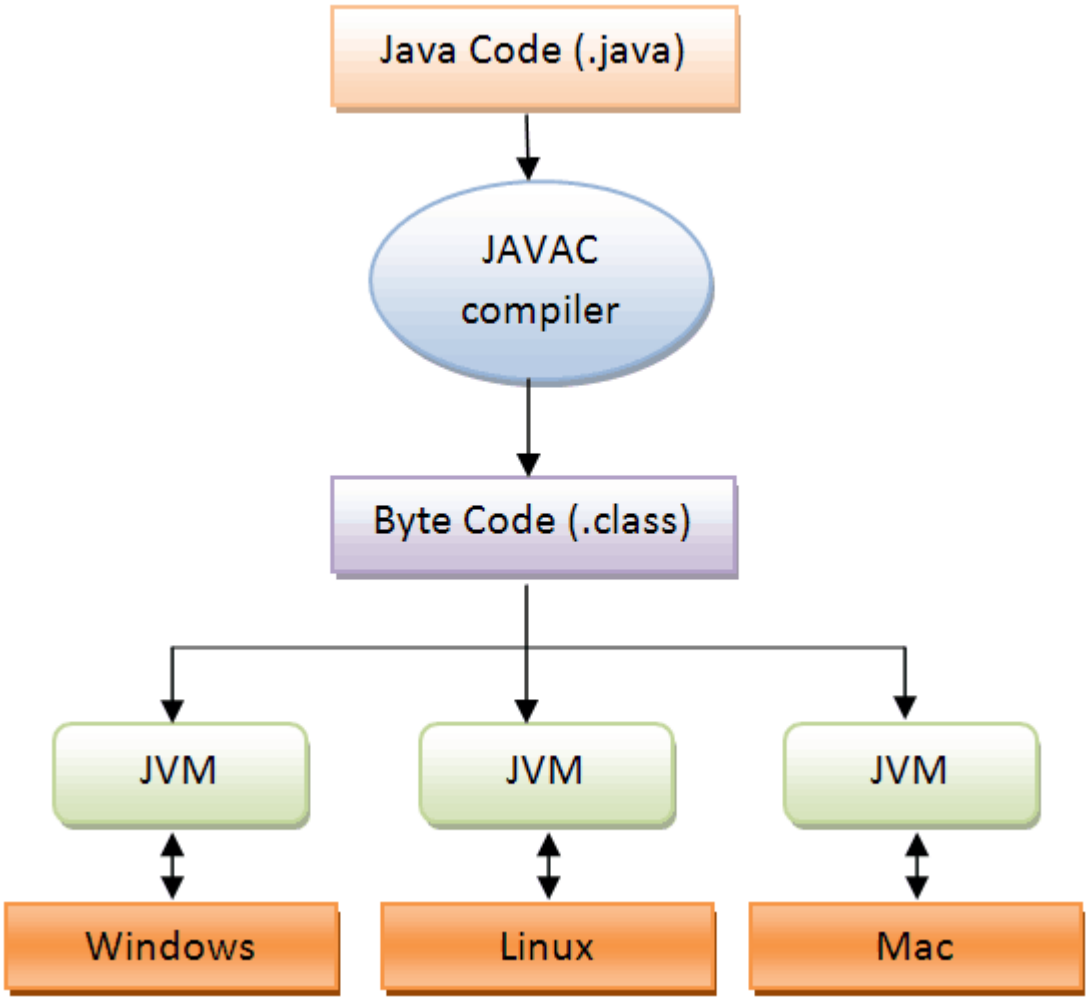
它们之间的关系如下：

- JVM 是 Java 虚拟机，是 Java 程序运行的环境。它负责将 Java 字节码（由 Java 编译器生成）解释或编译成机器码，并执行程序。JVM 提供了内存管理、垃圾回收、安全性等功能，使得 Java 程序具备跨平台性。
- JDK 是 Java 开发工具包，是开发 Java 程序所需的工具集合。它包含了 JVM、编译器（javac）、调试器（jdb）等开发工具，以及一系列的类库（如 Java 标准库和开发工具库）。JDK 提供了开发、编译、调试和运行 Java 程序所需的全部工具和环境。
- JRE 是 Java 运行时环境，是 Java 程序运行所需的最小环境。它包含了 JVM 和一组 Java 类库，用于支持 Java 程序的执行。JRE 不包含开发工具，只提供 Java 程序运行所需的运行环境。

JVM 和 Java 有啥区别?

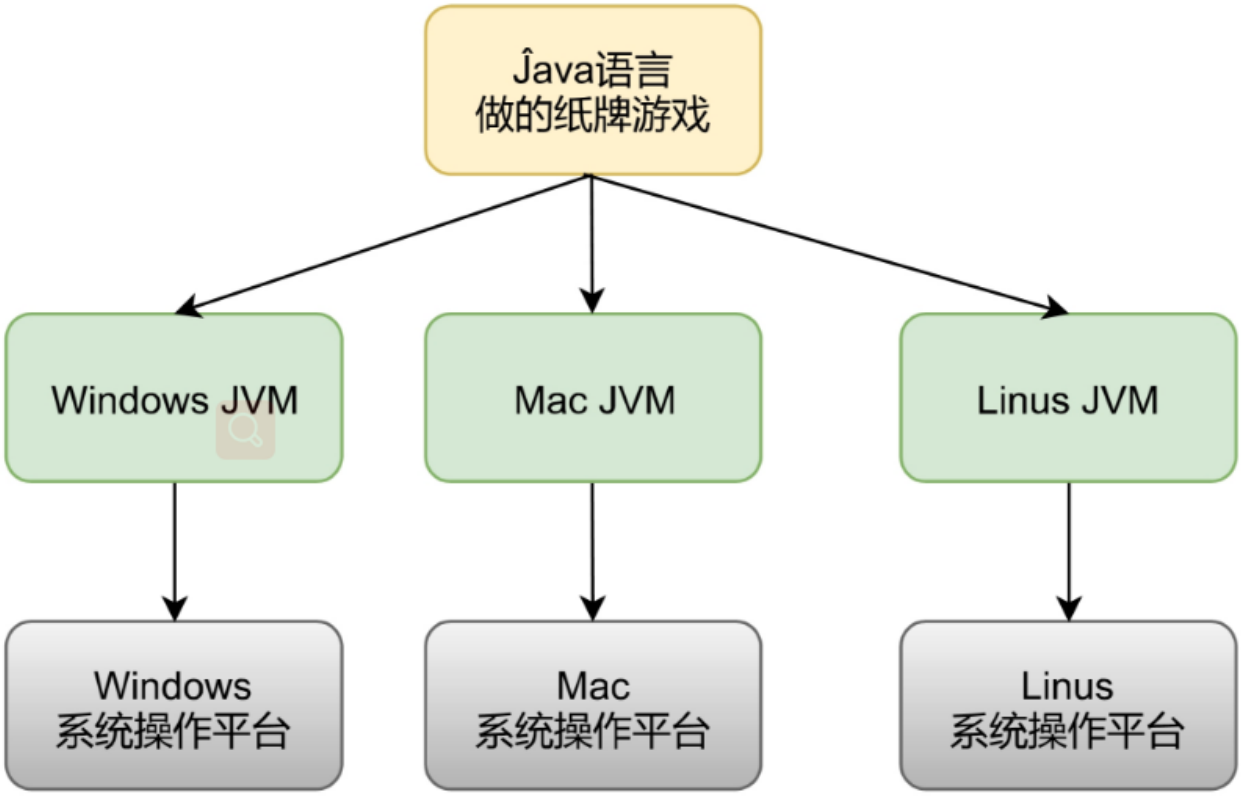
Java 是语言，JVM 是平台，一个是写代码的工具，一个是跑代码的环境，两者分工不同但相互配合。

简单来说，Java 是一门编程语言，就是我们平时写代码用的那个语言，像 String、List 这些都是 Java 语言提供的。而 JVM 呢，全称是 Java 虚拟机，它是用来运行 Java 程序的一个平台或者说运行环境。

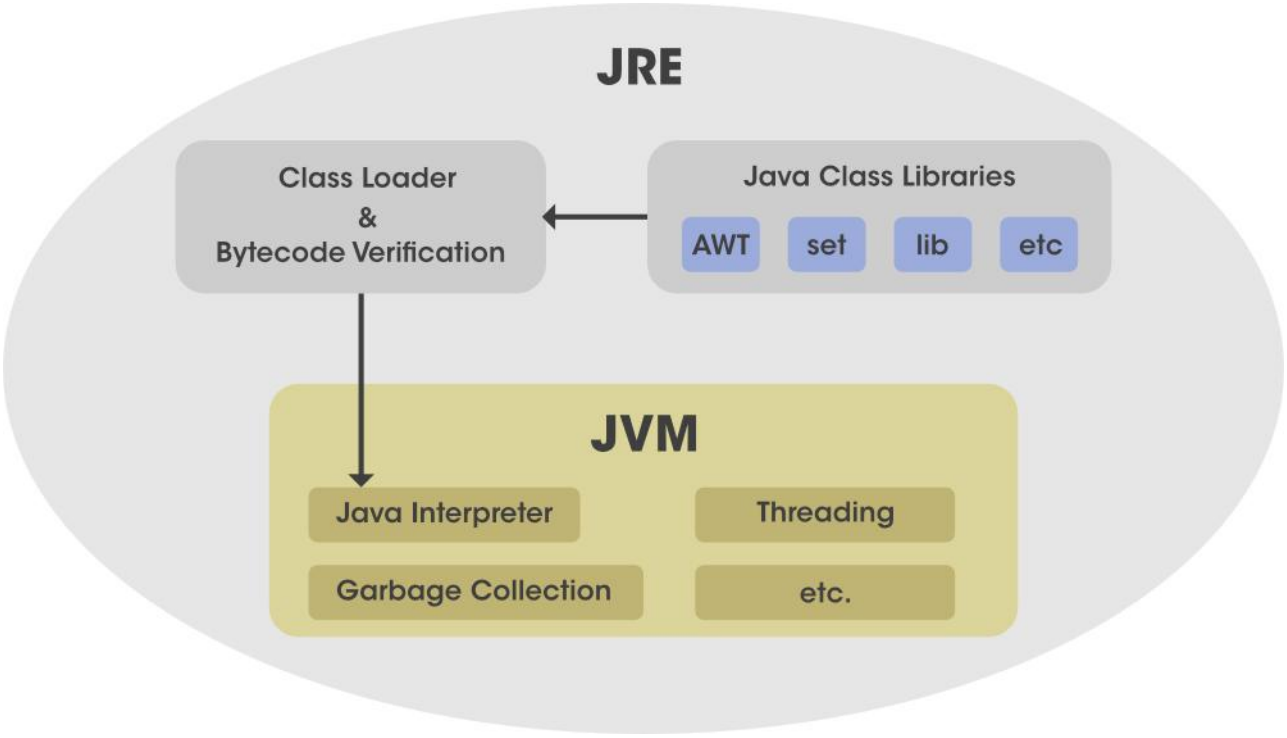


打个比方，Java 就像是我们写文章用的中文，而 JVM 就像是能够阅读和理解这篇文章的人。我们用 Java 语言写代码，但代码不能直接运行，得先通过编译器编译成字节码，也就是.class 文件，然后 JVM 再把这些字节码翻译成机器能懂的指令去执行。

这里面还有一个很关键的点，就是 Java 能够跨平台，也就是"一次编译，到处运行"。这个跨平台能力其实主要是 JVM 提供的。我们写好的 Java 代码编译成字节码后，这份字节码可以在 Windows 的 JVM 上运行，也可以在 Linux 的 JVM 上运行，还可以在 Mac 上运行。虽然底层操作系统不一样，但因为有 JVM 这一层，它帮我们屏蔽了这些差异，做好了适配。所以本质上是 JVM 依赖平台，但 Java 字节码不依赖平台。



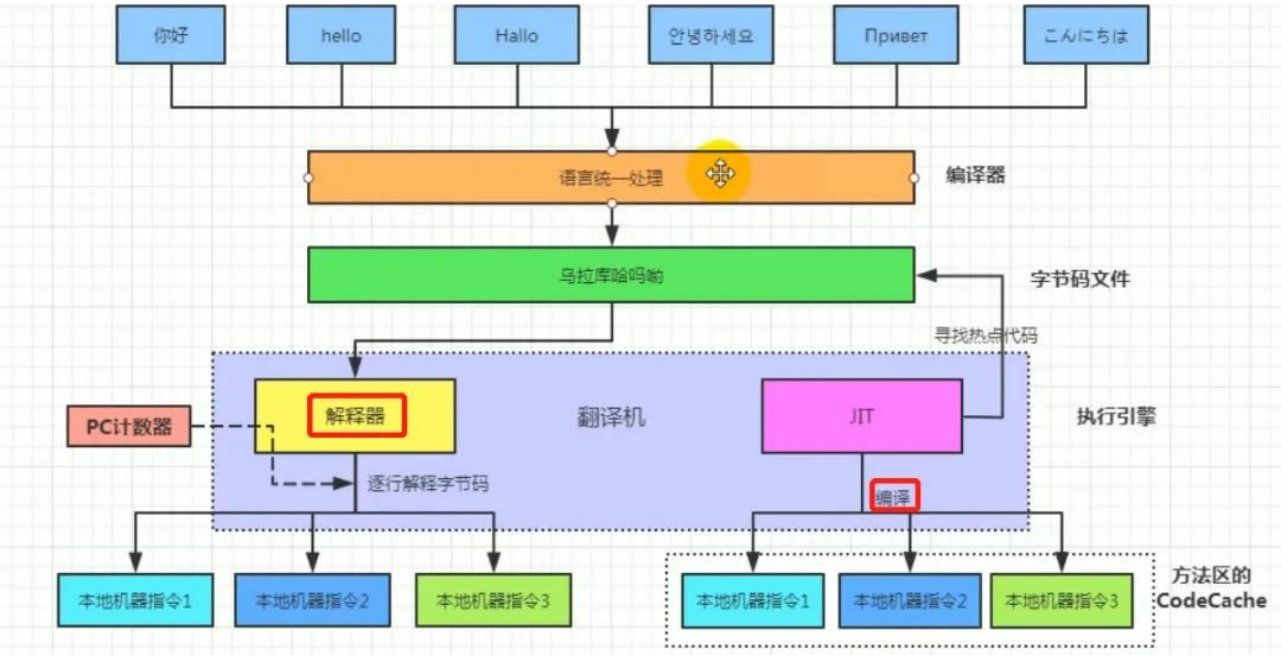
再简单说说它们和其他概念的关系。JDK 是开发工具包，里面有编译器、调试工具这些，也包含了 JRE。JRE 是运行环境，包含了 JVM 和 Java 的核心类库。所以关系就是 JDK 大于 JRE 大于 JVM，一个比一个小。如果你要开发 Java 程序就得装 JDK，如果只是运行别人写好的程序，装个 JRE 就够了。



还有一点，JVM 其实不只能跑 Java。因为 JVM 执行的是字节码，所以像 Kotlin、Scala 这些语言，虽然语法跟 Java 不一样，但编译后也是生成 Java 字节码，一样可以在 JVM 上运行。所以 JVM 现在已经不只是 Java 的专属了，而是整个 JVM 语言生态的基础平台。

为什么 Java 解释和编译都有?

首先在 Java 经过编译之后生成字节码文件，接下来进入 JVM 中，就有两个步骤编译和解释。 如下图：



编译性：

- Java 源代码首先被编译成字节码，JIT 会把编译过的机器码保存起来,以备下次使用。

解释性：

- JVM 中一个方法调用计数器，当累计计数大于一定值的时候，就使用 JIT 进行编译生成机器码文件。否则就是用解释器进行解释执行，然后字节码也是经过解释器进行解释运行的。

所以 Java 既是编译型也是解释性语言，默认采用的是解释器和编译器混合的模式。

jvm 是什么

JVM 是 java 虚拟机，主要工作是解释自己的指令集（即字节码）并映射到本地的 CPU 指令集和 OS 的系统调用。

JVM 屏蔽了与操作系统平台相关的信息，使得 Java 程序只需要生成在 Java 虚拟机上运行的目标代码（字节码），就可在多种平台上不加修改的运行，这也是 Java 能够**“一次编译，到处运行的”**原因。

编译型语言和解释型语言的区别?

编译型语言和解释型语言的区别在于：

- 编译型语言：在程序执行之前，整个源代码会被编译成机器码或者字节码，生成可执行文件。执行时直接运行编译后的代码，速度快，但跨平台性较差。
- 解释型语言：在程序执行时，逐行解释执行源代码，不生成独立的可执行文件。通常由解释器动态解释并执行代码，跨平台性好，但执行速度相对较慢。
- 典型的编译型语言如 C、C++，典型的解释型语言如 Python、JavaScript。

Python 和 Java 区别是什么?

- Java 是一种已编译的编程语言，Java 编译器将源代码编译为字节码，而字节码则由 Java 虚拟机执行
- python 是一种解释语言，翻译时会在执行程序的同时进行翻译。

值传递和引用传递的区别?

在 Java 中，参数传递只有**值传递**一种方式，不存在真正的 “引用传递”。但很多人会混淆这两个概念，核心区别在于传递的是 “值的副本” 还是 “引用的副本”。

值传递（Pass by Value）。传递的是实际值的副本，适用于**基本数据类型**（如 `int`、`char` 等），修改方法内的参数副本，**不会影响原变量的值**。例子：

```
public static void main(String[] args) {
    int num = 10;
    changeValue(num);
    System.out.println(num); // 输出 10（原变量未被修改）
}

public static void changeValue(int a) {
    a = 20; // 仅修改副本
}
```

引用传递的误解（本质仍是值传递）。对于对象（引用类型），传递的是**对象引用的副本**（而非对象本身）。

两个引用（原引用和副本）指向**同一个对象**，因此通过副本修改对象内部数据，**会影响原对象**。但如果修改副本的指向（如重新赋值），**不会影响原引用的指向**。示例：

```
public class Person {
    String name;
```

```
    Person(String name) { this.name = name; }
}

public static void main(String[] args) {
    Person p = new Person("Alice");
    changeName(p);
    System.out.println(p.name); // 输出 "Bob"（对象内部被修改）

    changeReference(p);
    System.out.println(p.name); // 仍输出 "Bob"（原引用指向未变）
}

// 修改对象内部数据
public static void changeName(Person obj) {
    obj.name = "Bob"; // 副本和原引用指向同一个对象
}

// 修改副本的指向（不影响原引用）
public static void changeReference(Person obj) {
    obj = new Person("Charlie"); // 副本指向新对象，原引用仍指向旧对象
}
```

简单来说，Java 中**所有参数传递都是值传递：**

- 基本类型传递 “值的副本”，修改副本不影响原值。
- 引用类型传递 “引用的副本”，通过副本可修改对象内容，但无法改变原引用的指向。

数据类型

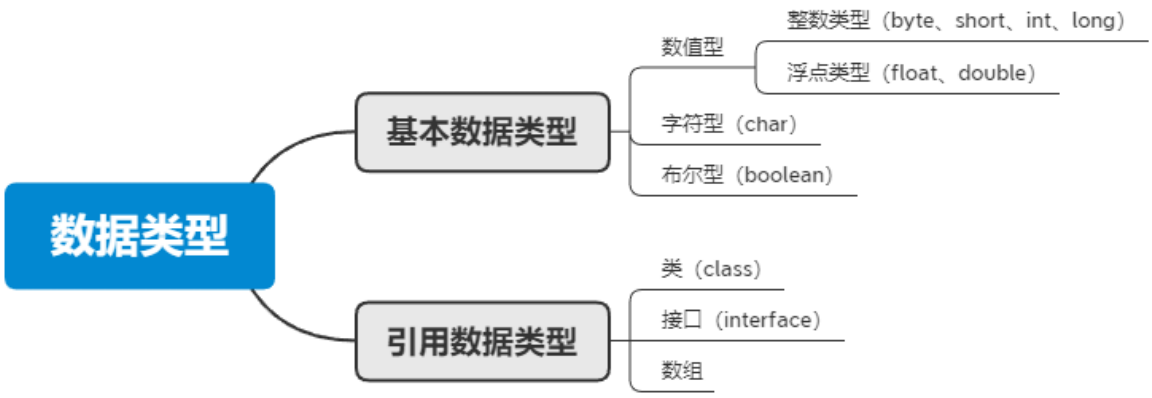
八种基本的数据类型

Java 支持数据类型分为两类： 基本数据类型和引用数据类型。

基本数据类型共有 8 种，可以分为三类：

- 数值型：整数类型（byte、short、int、long）和浮点类型（float、double）
- 字符型：char

- 布尔型：boolean



8 种基本数据类型的默认值、位数、取值范围，如下表所示：

数据类型	占用大小（字节）	位数	取值范围	默认值	描述
`byte`	1	8	-128（-2^7） 到 127（2^7 - 1）	0	是最小的整数类型，适合用于节省内存，例如在处理文件或网络流时存储小范围整数数据。
`short`	2	16	-32768（-2^15） 到 32767（2^15 - 1）	0	较少使用，通常用于在需要节省内存且数值范围在该区间的场景。

数据类型	占用大小（字节）	位数	取值范围	默认值	描述
`int`	4	32	-2147483648（-2^31） 到 2147483647（2^31 - 1）	0	最常用的整数类型，可满足大多数日常编程中整数计算的需求。
`long`	8	64	-9223372036854775808（-2^63） 到 9223372036854775807（2^63 - 1）	0L	用于表示非常大的整数，当 `int` 类型无法满足需求时使用，定义时数值后需加 `L` 或 `l`。
`float`	4	32	1.4E - 45 到 3.4028235E38	0.0f	单精度浮点数，用于表示小数，精度相对较低，定义时数值后需加 `F` 或 `f`。
`double`	8	64	4.9E - 324 到 1.7976931348623157E308	0.0d	双精度浮点数，精度比 `float` 高，是 Java 中表示小数的默认类型。
`char`	2	16	'\u0000'（0） 到 '\uffff'（65535）	'\u0000'	用于表示单个字符，采用 Unicode

数据类型	占用大小（字节）	位数	取值范围	默认值	描述
					编码，可表示各种语言的字符。
`boolean`	无明确字节大小（理论上 1 位）	无明确位数	`true` 或 `false`	`false`	用于逻辑判断，只有两个取值，常用于条件判断和循环控制等逻辑场景。

Float 和 Double 的最小值和最大值都是以科学记数法的形式输出的，结尾的“E+数字”表示 E 之前的数字要乘以 10 的多少倍。比如 3.14E3 就是 3.14×1000=3140，3.14E-3 就是 3.14/1000=0.00314。

注意一下几点：

- Java 八种基本数据类型的字节数：1 字节(byte、boolean)、 2 字节(short、char)、4 字节(int、float)、8 字节(long、double)
- 浮点数的默认类型为 double（如果需要声明一个常量为 float 型，则必须要在末尾加上 f 或 F）
- 整数的默认类型为 int（声明 Long 型在末尾加上 l 或者 L）
- 八种基本数据类型的包装类：除了 char 的是 Character、int 类型的是 Integer，其他都是首字母大写
- char 类型是无符号的，不能为负，所以是 0 开始的

int 和 long 是多少位，多少字节的?

- `int` 类型是 32 位（bit），占 4 个字节（byte），int 是有符号整数类型，其取值范围是从 -2^31 到 2^31-1 。例如，在一个简单的计数器程序中，如果使用 `int` 类型来存储计数值，它可以表示的最大正数是 2,147,483,647。如果计数值超过这个范围，就会发生溢出，导致结果不符合预期。

- `long` 类型是 64 位，占 8 个字节，`long` 类型也是有符号整数类型，它的取值范围是从 -2^63 到 2^63 -1 ，在处理较大的整数数值时，果 `int` 类型的取值范围不够，就需要使用 `long` 类型。例如，在一个文件传输程序中，文件的大小可能会很大，使用 `int` 类型可能无法准确表示，而 `long` 类型就可以很好地处理这种情况。

long 和 int 可以互转吗？

可以的，Java 中的 `long` 和 `int` 可以相互转换。由于 `long` 类型的范围比 `int` 类型大，因此将 `int` 转换为 `long` 是安全的，而将 `long` 转换为 `int` 可能会导致数据丢失或溢出。

将 `int` 转换为 `long` 可以通过直接赋值或强制类型转换来实现。例如：

```
int intValue = 10;
long longValue = intValue; // 自动转换，安全的
```

将 `long` 转换为 `int` 需要使用强制类型转换，但需要注意潜在的数据丢失或溢出问题。



例如：

```
long longValue = 100L;
int intValue = (int) longValue; // 强制类型转换，可能会有数据丢失或溢出
```

在将 `long` 转换为 `int` 时，如果 `longValue` 的值超出了 `int` 类型的范围，转换结果将是截断后的低位部分。因此，在进行转换之前，建议先检查 `longValue` 的值是否在 `int` 类型的范围内，以避免数据丢失或溢出的问题。

数据类型转换方式你知道哪些?

- 自动类型转换（隐式转换）：当目标类型的范围大于源类型时，Java 会自动将源类型转换为目标类型，不需要显式的类型转换。例如，将 `int` 转换为 `long`、将 `float` 转换为 `double` 等。

- 强制类型转换（显式转换）：当目标类型的范围小于源类型时，需要使用强制类型转换将源类型转换为目标类型。这可能导致数据丢失或溢出。例如，将 `long` 转换为 `int`、将 `double` 转换为 `int` 等。语法为：目标类型 变量名 =（目标类型）源类型。
- 字符串转换：Java 提供了将字符串表示的数据转换为其他类型数据的方法。例如，将字符串转换为整型 `int`，可以使用 `Integer.parseInt()`方法；将字符串转换为浮点型 `double`，可以使用 `Double.parseDouble()`方法等。
- 数值之间的转换：Java 提供了一些数值类型之间的转换方法，如将整型转换为字符型、将字符型转换为整型等。这些转换方式可以通过类型的包装类来实现，例如 `Character` 类、`Integer` 类等提供了相应的转换方法。

类型互转会出现什么问题吗?

> 基本数据类型转换的问题

当把小范围数据类型赋值给大范围数据类型时，Java 会自动进行类型转换，这种转换一般是安全的。

```
int num = 100;
long bigNum = num; // 自动将 int 转换为 long
```

但是大范围数据类型赋值给小范围数据类型时，会发生数据数据溢出或者精度损失的问题。

- 数据溢出：如果大范围数据类型赋值给小范围数据类型时，当目标类型无法容纳原数据时，就会发生数据溢出。比如下面，byte 类型的取值范围是 - 128 到 127。300 的二进制表示为 `00000001 00101100`，强制转换为 byte 类型时，会丢弃高位字节，只保留低位的 8 位 `00101100`，其十进制值为 44。

```
int largeNum = 300;
byte b = (byte) largeNum; // b 的值为 44
```

- 精度损失：在进行浮点数类型的转换时，可能会发生精度损失。由于浮点数的表示方式不同，将一个单精度浮点数(`float`)转换为双精度浮点数(`double`)时，精度可能会损失，如果 double 转换为 int 也会发生精度损失的问题，如下：

```
double d = 3.14;
int i = (int) d; // i 的值为 3，小数部分 0.14 被舍弃
```

> 对象引用转换的问题

向上转型是自动进行的，而且是安全的，如下：

```
class Animal {}
class Dog extends Animal {}


Dog dog = new Dog();
Animal animal = dog; // 自动向上转型
```

但是向下转型需要手动进行，并且存在风险。如果父类对象实际上并不是目标子类的实例，在转型时就会抛出异常：

```
Animal animal = new Animal();
Dog dog = (Dog) animal; // 运行时抛出 ClassCastException
```

原因是 Java 的对象在运行时会记录其真实类型，当进行向下转型时，Java 会检查对象的实际类型是否与目标类型兼容。如果不兼容，就会抛出 `ClassCastException`。

解决方式是需要使用 `instanceof` 检查：

```
if (animal instanceof Dog) {
    Dog dog = (Dog) animal; // 只有确认 animal 是 Dog 的实例时才进行转型
}
```

为什么用 bigDecimal 不用 double？

double 会出现精度丢失的问题，double 执行的是二进制浮点运算，二进制有些情况下不能准确的表示一个小数，就像十进制不能准确的表示 1/3(1/3=0.3333...)，也就是说二进制表示小数的时候只能够表示能够用 1/(2^n)的和的任意组合，但是 0.1 不能够精确表示，因为它不能够表示成为 1/(2^n)的和的形式。

比如：

```
System.out.println(0.05 + 0.01);
System.out.println(1.0 - 0.42);
System.out.println(4.015 * 100);
System.out.println(123.3 / 100);
```

输出：

```
0.060000000000000005
0.5800000000000001
401.49999999999994
1.2329999999999999
```

可以看到在 Java 中进行浮点数运算的时候，会出现丢失精度的问题。那么我们如果在进行商品价格计算的时候，就会出现问题。很有可能造成我们手中有 0.06 元，却无法购买一个 0.05 元和一个 0.01 元的商品。因为如上所示，他们两个的总和为 0.060000000000000005。这无疑是一个很严重的问题，尤其是当电商网站的并发量上去的时候，出现的问题将是巨大的。可能会导致无法下单，或者对账出现问题。

而 Decimal 是精确计算 ， 所以一般牵扯到金钱的计算 ， 都使用 Decimal。

```
import java.math.BigDecimal;

public class BigDecimalExample {
    public static void main(String[] args) {
        BigDecimal num1 = new BigDecimal("0.1");
        BigDecimal num2 = new BigDecimal("0.2");

        BigDecimal sum = num1.add(num2);
        BigDecimal product = num1.multiply(num2);

        System.out.println("Sum: " + sum);
        System.out.println("Product: " + product);
    }
}

//输出
Sum: 0.3
Product: 0.02
```

在上述代码中，我们创建了两个 `BigDecimal` 对象 `num1` 和 `num2`，分别表示 0.1 和 0.2 这两个十进制数。然后，我们使用 `add()`方法计算它们的和，并使用 `multiply()`方法计算它们的乘积。最后，我们通过 `System.out.println()`打印结果。

这样的使用 `BigDecimal` 可以确保精确的十进制数值计算，避免了使用 `double` 可能出现的舍入误差。需要注意的是，在创建 `BigDecimal` 对象时，应该使用字符串作为参数，而不是直接使用浮点数值，以避免浮点数精度丢失。

装箱和拆箱是什么?

装箱（Boxing）和拆箱（Unboxing）是将基本数据类型和对应的包装类之间进行转换的过程。

```
Integer i = 10;  //装箱
int n = i;   //拆箱
```

自动装箱主要发生在两种情况，一种是赋值时，另一种是在方法调用的时候。

> 赋值时

这是最常见的一种情况，在 Java 1.5 以前我们需要手动地进行转换才行，而现在所有的转换都是由编译器来完成。

```
//before autoboxing
Integer iObject = Integer.valueOf(3);
Int iPrimitive = iObject.intValue()

//after java5
Integer iObject = 3; //autobxing - primitive to wrapper conversion
int iPrimitive = iObject; //unboxing - object to primitive conversion
```

> 方法调用时

当我们在方法调用时，我们可以传入原始数据值或者对象，同样编译器会帮我们进行转换。

```
public static Integer show(Integer iParam){
   System.out.println("autoboxing example - method invocation i: " + iParam);
   return iParam;
}

//autoboxing and unboxing in method invocation
show(3); //autoboxing
int result = show(3); //unboxing because return type of method is Integer
```

show 方法接受 Integer 对象作为参数，当调用 `show(3)`时，会将 int 值转换成对应的 Integer 对象，这就是所谓的自动装箱，show 方法返回 Integer 对象，而 `int result = show(3);`中 result 为 int 类型，所以这时候发生自动拆箱操作，将 show 方法的返回的 Integer 对象转换成 int 值。

> 自动装箱的弊端

自动装箱有一个问题，那就是在一个循环中进行自动装箱操作的情况，如下面的例子就会创建多余的对象，影响程序的性能。

```
Integer sum = 0; for(int i=1000; i<5000; i++){   sum+=i; }
```

上面的代码 `sum+=i` 可以看成 `sum = sum + i`，但是+这个操作符不适用于 Integer 对象，首先 sum 进行自动拆箱操作，进行数值相加操作，最后发生自动装箱操作转换成 Integer 对象。其内部变化如下

```
int result = sum.intValue() + i; Integer sum = new Integer(result);
```

由于我们这里声明的 sum 为 Integer 类型，在上面的循环中会创建将近 4000 个无用的 Integer 对象，在这样庞大的循环中，会降低程序的性能并且加重了垃圾回收的工作量。因此在我们编程时，需要注意到这一点，正确地声明变量类型，避免因为自动装箱引起的性能问题。

Java 为什么要有 Integer?

Integer 对应是 int 类型的包装类，就是把 int 类型包装成 Object 对象，对象封装有很多好处，可以把属性也就是数据跟处理这些数据的方法结合在一起，比如 Integer 就有 parseInt()等方法来专门处理 int 型相关的数据。

另一个非常重要的原因就是在 Java 中绝大部分方法或类都是用来处理类类型对象的，如 ArrayList 集合类就只能以类作为他的存储对象，而这时如果想把一个 int 型的数据存入 list 是不可能的，必须把它包装成类，也就是 Integer 才能被 List 所接受。所以 Integer 的存在是很必要的。

> 泛型中的应用

在 Java 中，泛型只能使用引用类型，而不能使用基本类型。因此，如果要在泛型中使用 int 类型，必须使用 Integer 包装类。例如，假设我们有一个列表，我们想要将其元素排序，并将排序结果存储在一个新的列表中。如果我们使用基本数据类型 int，无法直接使用 Collections.sort()方法。但是，如果我们使用 Integer 包装类，我们就可以轻松地使用 Collections.sort()方法。

```
List<Integer> list = new ArrayList<>();
```

```
list.add(3);
list.add(1);
list.add(2);
Collections.sort(list);
System.out.println(list);
```

> 转换中的应用

在 Java 中，基本类型和引用类型不能直接进行转换，必须使用包装类来实现。例如，将一个 int 类型的值转换为 String 类型，必须首先将其转换为 Integer 类型，然后再转换为 String 类型。

```
int i = 10;
Integer integer = new Integer(i);
String str = integer.toString();
System.out.println(str);
```

> 集合中的应用

Java 集合中只能存储对象，而不能存储基本数据类型。因此，如果要将 int 类型的数据存储在集合中，必须使用 Integer 包装类。例如，假设我们有一个列表，我们想要计算列表中所有元素的和。如果我们使用基本数据类型 int，我们需要使用一个循环来遍历列表，并将每个元素相加。但是，如果我们使用 Integer 包装类，我们可以直接使用 stream()方法来计算所有元素的和。

```
List<Integer> list = new ArrayList<>();
list.add(3);
list.add(1);
list.add(2);
int sum = list.stream().mapToInt(Integer::intValue).sum();
System.out.println(sum);
```

Integer 相比 int 有什么优点?

int 是 Java 中的原始数据类型，而 Integer 是 int 的包装类。

Integer 和 int 的区别：

- 基本类型和引用类型：首先，int 是一种基本数据类型，而 Integer 是一种引用类型。基本数据类型是 Java 中最基本的数据类型，它们是预定义的，不需要实例化就可以使用。而引用类型则需要通过实例化对象来使用。这意味着，使用 int 来存储一个整数时，不需要任何额

外的内存分配，而使用 Integer 时，必须为对象分配内存。在性能方面，基本数据类型的操作通常比相应的引用类型快。

- 自动装箱和拆箱：其次，Integer 作为 int 的包装类，它可以实现自动装箱和拆箱。自动装箱是指将基本类型转化为相应的包装类类型，而自动拆箱则是将包装类类型转化为相应的基本类型。这使得 Java 程序员更加方便地进行数据类型转换。例如，当我们需要将 int 类型的值赋给 Integer 变量时，Java 可以自动地将 int 类型转换为 Integer 类型。同样地，当我们需要将 Integer 类型的值赋给 int 变量时，Java 可以自动地将 Integer 类型转换为 int 类型。
- 空指针异常：另外，int 变量可以直接赋值为 0，而 Integer 变量必须通过实例化对象来赋值。如果对一个未经初始化的 Integer 变量进行操作，就会出现空指针异常。这是因为它被赋予了 null 值，而 null 值是无法进行自动拆箱的。

那为什么还要保留 int 类型?

包装类是引用类型，对象的引用和对象本身是分开存储的，而对于基本类型数据，变量对应的内存块直接存储数据本身。

因此，基本类型数据在读写效率方面，要比包装类高效。除此之外，在 64 位 JVM 上，在开启引用压缩的情况下，一个 Integer 对象占用 16 个字节的内存空间，而一个 int 类型数据只占用 4 字节的内存空间，前者对空间的占用是后者的 4 倍。

也就是说，不管是读写效率，还是存储效率，基本类型都比包装类高效。

说一下 integer 的缓存

Java 的 Integer 类内部实现了一个静态缓存池，用于存储特定范围内的整数值对应的 Integer 对象。

默认情况下，这个范围是-128 至 127。当通过 Integer.valueOf(int)方法创建一个在这个范围内的整数对象时，并不会每次都生成新的对象实例，而是复用缓存中的现有对象，会直接从内存中取出，不需要新建一个对象。

面向对象

怎么理解面向对象？简单说说封装继承多态

面向对象是一种编程范式，它**将现实世界中的事物抽象为对象**，对象具有属性（称为字段或属性）和行为（称为方法）。面向对象编程的设计思想是以对象为中心，通过对象之间的交互来完成程序的功能，具有灵活性和可扩展性，通过封装和继承可以更好地应对需求变化。

Java 面向对象的三大特性包括：**封装、继承、多态**：

- **封装**：封装是指将对象的属性（数据）和行为（方法）结合在一起，对外隐藏对象的内部细节，仅通过对象提供的接口与外界交互。封装的目的是增强安全性和简化编程，使得对象更加独立。
- **继承**：继承是一种可以使得子类自动共享父类数据结构和方法的机制。它是代码复用的重要手段，通过继承可以建立类与类之间的层次关系，使得结构更加清晰。
- **多态**：多态是指允许不同类的对象对同一消息作出响应。即同一个接口，使用不同的实例而执行不同操作。多态性可以分为编译时多态（重载）和运行时多态（重写）。它使得程序具有良好的灵活性和扩展性。

多态体现在哪几个方面?

多态在面向对象编程中可以体现在以下几个方面：

- **方法重载**：
  - 方法重载是指同一类中可以有多个同名方法，它们具有不同的参数列表（参数类型、数量或顺序不同）。虽然方法名相同，但根据传入的参数不同，编译器会在编译时确定调用哪个方法。
  - 示例：对于一个 `add` 方法，可以定义为 `add(int a, int b)` 和 `add(double a, double b)`。
- **方法重写**：
  - 方法重写是指子类能够提供对父类中同名方法的具体实现。在运行时，JVM 会根据对象的实际类型确定调用哪个版本的方法。这是实现多态的主要方式。
  - 示例：在一个动物类中，定义一个 `sound` 方法，子类 `Dog` 可以重写该方法以实现 `bark`，而 `Cat` 可以实现 `meow`。
- **接口与实现**：
  - 多态也体现在接口的使用上，多个类可以实现同一个接口，并且用接口类型的引用来调用这些类的方法。这使得程序在面对不同具体实现时保持一贯的调用方式。

    - 示例：多个类（如 `Dog`, `Cat`）都实现了一个 `Animal` 接口，当用 `Animal` 类型的引用来调用 `makeSound` 方法时，会触发对应的实现。
- **向上转型和向下转型：**
    - 在 Java 中，可以使用父类类型的引用指向子类对象，这是向上转型。通过这种方式，可以在运行时期采用不同的子类实现。
    - 向下转型是将父类引用转回其子类类型，但在执行前需要确认引用实际指向的对象类型以避免 `ClassCastException`。

多态解决了什么问题?

多态是指子类可以替换父类，在实际的代码运行过程中，调用子类的方法实现。多态这种特性也需要编程语言提供特殊的语法机制来实现，比如继承、接口类。

多态可以提高代码的扩展性和复用性，是很多设计模式、设计原则、编程技巧的代码实现基础。比如策略模式、基于接口而非实现编程、依赖倒置原则、里式替换原则、利用多态去掉冗长的 if-else 语句等等

面向对象的设计原则你知道有哪些吗

面向对象编程中的六大原则：

- **单一职责原则（SRP）**：一个类应该只有一个引起它变化的原因，即一个类应该只负责一项职责。例子：考虑一个员工类，它应该只负责管理员工信息，而不应负责其他无关工作。
- **开放封闭原则（OCP）**：软件实体应该对扩展开放，对修改封闭。例子：通过制定接口来实现这一原则，比如定义一个图形类，然后让不同类型的图形继承这个类，而不需要修改图形类本身。
- **里氏替换原则（LSP）**：子类对象应该能够替换掉所有父类对象。例子：一个正方形是一个矩形，但如果修改一个矩形的高度和宽度时，正方形的行为应该如何改变就是一个违反里氏替换原则的例子。
- **接口隔离原则（ISP）**：客户端不应该依赖那些它不需要的接口，即接口应该小而专。例子：通过接口抽象层来实现底层和高层模块之间的解耦，比如使用依赖注入。
- **依赖倒置原则（DIP）**：高层模块不应该依赖低层模块，二者都应该依赖于抽象；抽象不应该依赖于细节，细节应该依赖于抽象。例子：如果一个公司类包含部门类，应该考虑使用合成/聚合关系，而不是将公司类继承自部门类。

- **最少知识原则 (Law of Demeter)**：一个对象应当对其他对象有最少的了解，只与其直接的朋友交互。

重载与重写有什么区别?

- 重载（Overloading）指的是在同一个类中，可以有多个同名方法，它们具有不同的参数列表（参数类型、参数个数或参数顺序不同），编译器根据调用时的参数类型来决定调用哪个方法。
- 重写（Overriding）指的是子类可以重新定义父类中的方法，方法名、参数列表和返回类型必须与父类中的方法一致，通过@override 注解来明确表示这是对父类方法的重写。

重载是指在同一个类中定义多个同名方法，而重写是指子类重新定义父类中的方法。

抽象类和普通类区别?

- 实例化：普通类可以直接实例化对象，而抽象类不能被实例化，只能被继承。
- 方法实现：普通类中的方法可以有具体的实现，而抽象类中的方法可以有实现也可以没有实现。
- 继承：一个类可以继承一个普通类，而且可以继承多个接口；而一个类只能继承一个抽象类，但可以同时实现多个接口。
- 实现限制：普通类可以被其他类继承和使用，而抽象类一般用于作为基类，被其他类继承和扩展使用。

Java 抽象类和接口的区别是什么?

两者的特点：

- 抽象类用于描述类的共同特性和行为，可以有成员变量、构造方法和具体方法。适用于有明显继承关系的场景。
- 接口用于定义行为规范，可以多实现，只能有常量和抽象方法（Java 8 以后可以有默认方法和静态方法）。适用于定义类的能力或功能。

两者的区别：

- 实现方式：实现接口的关键字为 implements，继承抽象类的关键字为 extends。一个类可以实现多个接口，但一个类只能继承一个抽象类。所以，使用接口可以间接地实现多重继承。
- 方法方式：接口只有定义，不能有方法的实现，java 1.8 中可以定义 default 方法体，而抽象类可以有定义与实现，方法可在抽象类中实现。

- 访问修饰符：接口成员变量默认为 public static final，必须赋初值，不能被修改；其所有的成员方法都是 public、abstract 的。抽象类中成员变量默认 default，可在子类中被重新定义，也可被重新赋值；抽象方法被 abstract 修饰，不能被 private、static、synchronized 和 native 等修饰，必须以分号结尾，不带花括号。
- 变量：抽象类可以包含实例变量和静态变量，而接口只能包含常量（即静态常量）。

抽象类能加 final 修饰吗?

不能，Java 中的抽象类是用来被继承的，而 final 修饰符用于禁止类被继承或方法被重写，因此，抽象类和 final 修饰符是互斥的，不能同时使用。

接口里面可以定义哪些方法?

- **抽象方法**

抽象方法是接口的核心部分，所有实现接口的类都必须实现这些方法。抽象方法默认是 public 和 abstract，这些修饰符可以省略。

```
public interface Animal {
    void makeSound();
}
```

- **默认方法**

默认方法是在 Java 8 中引入的，允许接口提供具体实现。实现类可以选择重写默认方法。

```
public interface Animal {
    void makeSound();

    default void sleep() {
        System.out.println("Sleeping...");
    }
}
```

- **静态方法**

静态方法也是在 Java 8 中引入的，它们属于接口本身，可以通过接口名直接调用，而不需要实现类的对象。

```
public interface Animal {
```

```
    void makeSound();

    static void staticMethod() {
        System.out.println("Static method in interface");
    }
}
```

- **私有方法**

私有方法是在 Java 9 中引入的，用于在接口中为默认方法或其他私有方法提供辅助功能。这些方法不能被实现类访问，只能在接口内部使用。

```
public interface Animal {
    void makeSound();

    default void sleep() {
        System.out.println("Sleeping...");
        logSleep();
    }

    private void logSleep() {
        System.out.println("Logging sleep");
    }
}
```

```
public interface Animal {
    void makeSound();
}
```

抽象类可以被实例化吗?

在 Java 中，抽象类本身不能被实例化。

这意味着不能使用 `new` 关键字直接创建一个抽象类的对象。抽象类的存在主要是为了被继承，它通常包含一个或多个抽象方法（由 `abstract` 关键字修饰且无方法体的方法），这些方法需要在子类中被实现。

抽象类可以有构造器，这些构造器在子类实例化时会被调用，以便进行必要的初始化工作。然而，这个过程并不是直接实例化抽象类，而是创建了子类的实例，间接地使用了抽象类的构造器。

例如：

```
public abstract class AbstractClass {
    public AbstractClass() {
        // 构造器代码
    }

    public abstract void abstractMethod();
}

public class ConcreteClass extends AbstractClass {
    public ConcreteClass() {
        super(); // 调用抽象类的构造器
    }

    @Override
    public void abstractMethod() {
        // 实现抽象方法
    }
}

// 下面的代码可以运行
ConcreteClass obj = new ConcreteClass();
```

在这个例子中，`ConcreteClass` 继承了 `AbstractClass` 并实现了抽象方法 `abstractMethod()`。当我们创建 `ConcreteClass` 的实例时，`AbstractClass` 的构造器被调用，但这并不意味着 `AbstractClass` 被实例化；实际上，我们创建的是 `ConcreteClass` 的一个对象。

简而言之，抽象类不能直接实例化，但通过继承抽象类并实现所有抽象方法的子类是可以被实例化的。

接口可以包含构造函数吗?

在接口中，不可以有构造方法,在接口里写入构造方法时，编译器提示：Interfaces cannot have constructors，因为接口不会有自己的实例的，所以不需要有构造函数。

为什么呢？构造函数就是初始化 class 的属性或者方法，在 new 的一瞬间自动调用，那么问题来了 Java 的接口，都不能 new 那么要构造函数干嘛呢？根本就没法调用

解释 Java 中的静态变量和静态方法

在 Java 中，静态变量和静态方法是与类本身关联的，而不是与类的实例（对象）关联。它们在内存中只存在一份，可以被类的所有实例共享。

> 静态变量

静态变量（也称为类变量）是在类中使用 `static` 关键字声明的变量。它们属于类而不是任何具体的对象。主要的特点：

- **共享性**：所有该类的实例共享同一个静态变量。如果一个实例修改了静态变量的值，其他实例也会看到这个更改。
- **初始化**：静态变量在类被加载时初始化，只会对其进行一次分配内存。
- **访问方式**：静态变量可以直接通过类名访问，也可以通过实例访问，但推荐使用类名。

示例：

```
public class MyClass {
    static int staticVar = 0; // 静态变量

    public MyClass() {
        staticVar++; // 每创建一个对象，静态变量自增
    }

    public static void printStaticVar() {
        System.out.println("Static Var: " + staticVar);
    }
}

// 使用示例
MyClass obj1 = new MyClass();
MyClass obj2 = new MyClass();
```

```
MyClass.printStaticVar(); // 输出 Static Var: 2
```

> 静态方法

静态方法是在类中使用 `static` 关键字声明的方法。类似于静态变量，静态方法也属于类，而不是任何具体的对象。主要的特点：

- **无实例依赖**：静态方法可以在没有创建类实例的情况下调用。对于静态方法来说，不能直接访问非静态的成员变量或方法，因为静态方法没有上下文的实例。
- **访问静态成员**：静态方法可以直接调用其他静态变量和静态方法，但不能直接访问非静态成员。
- **多态性**：静态方法不支持重写（Override），但可以被隐藏（Hide）。

```
public class MyClass {
    static int count = 0;

    // 静态方法
    public static void incrementCount() {
        count++;
    }

    public static void displayCount() {
        System.out.println("Count: " + count);
    }
}

// 使用示例
MyClass.incrementCount(); // 调用静态方法
MyClass.displayCount();   // 输出 Count: 1
```

> 使用场景

- **静态变量**：常用于需要在所有对象间共享的数据，如计数器、常量等。
- **静态方法**：常用于助手方法（utility methods）、获取类级别的信息或者是没有依赖于实例的数据处理。

非静态内部类和静态内部类的区别?

区别包括：

- 非静态内部类依赖于外部类的实例，而静态内部类不依赖于外部类的实例。
- 非静态内部类可以访问外部类的实例变量和方法，而静态内部类只能访问外部类的静态成员。
- 非静态内部类不能定义静态成员，而静态内部类可以定义静态成员。
- 非静态内部类在外部类实例化后才能实例化，而静态内部类可以独立实例化。
- 非静态内部类可以访问外部类的私有成员，而静态内部类不能直接访问外部类的私有成员，需要通过实例化外部类来访问。

非静态内部类可以直接访问外部方法，编译器是怎么做到的?

非静态内部类可以直接访问外部方法是因为编译器在生成字节码时会为非静态内部类维护一个指向外部类实例的引用。

这个引用使得非静态内部类能够访问外部类的实例变量和方法。编译器会在生成非静态内部类的构造方法时，将外部类实例作为参数传入，并在内部类的实例化过程中建立外部类实例与内部类实例之间的联系，从而实现直接访问外部方法的功能。

关键字

Java 中 final 作用是什么?

`final` 关键字主要有以下三个方面的作用：用于修饰类、方法和变量。

- 修饰类：当 `final` 修饰一个类时，表示这个类不能被继承，是类继承体系中的最终形态。例如，Java 中的 `String` 类就是用 `final` 修饰的，这保证了 `String` 类的不可变性和安全性，防止其他类通过继承来改变 `String` 类的行为和特性。
- 修饰方法：用 `final` 修饰的方法不能在子类中被重写。比如，`java.lang.Object` 类中的 `getClass` 方法就是 `final` 的，因为这个方法的行为是由 Java 虚拟机底层实现来保证的，不应该被子类修改。
- 修饰变量：当 `final` 修饰基本数据类型的变量时，该变量一旦被赋值就不能再改变。例如，`final int num = 10;`，这里的 `num` 就是一个常量，不能再对其进行重新赋值操作，否则会导致编译错误。对于引用数据类型，`final` 修饰意味着这个引用变量不能再指向其他对象，但对象本身的内容是可以改变的。例如，`final StringBuilder sb = new StringBuilder("Hello");`，

不能让 `sb` 再指向其他 `StringBuilder` 对象，但可以通过 `sb.append(" World");`来修改字符串的内容。

Java 中 static 的作用是什么?

`static` 关键字主要用于修饰类的成员（变量、方法、代码块）和内部类，其核心作用是**将成员与类本身关联，而非与类的实例（对象）关联**。具体作用如下：

> 1、修饰变量

被 `static` 修饰的变量属于类本身，而非类的某个实例。所有对象共享同一份静态变量，内存中只存在一份副本。可以通过 `类名.变量名` 直接访问，无需创建对象（也可通过对象访问，但不推荐）。

通常用于存储所有对象共享的数据，如常量、计数器等。

```
public class Student {
    // 静态变量（所有学生共享同一个学校名称）
    public static String schoolName = "阳光中学";
    // 实例变量（每个学生有自己的姓名）
    private String name;
}

// 访问静态变量
public class Test {
    public static void main(String[] args) {
        System.out.println(Student.schoolName); // 直接通过类名访问
    }
}
```

> 2、修饰方法

静态方法属于类，不属于任何实例，因此**不能直接访问类中的非静态成员（变量 / 方法）**（因为非静态成员依赖于对象存在），但可以访问静态成员。通过 `类名.方法名` 直接调用，无需创建对象。

通常用于工具类方法（如 `Math.random()`）、工厂方法等，不需要依赖对象状态即可完成操作。

```
public class MathUtils {
    // 静态方法（无需创建对象即可调用）
```

```
    public static int add(int a, int b) {
        return a + b;
    }
}

// 调用静态方法
public class Test {
    public static void main(String[] args) {
        int result = MathUtils.add(2, 3); // 直接通过类名调用
    }
}
```

3、修饰代码块

静态代码块在**类加载时执行**，且只执行一次（优于对象构造方法），用于初始化静态变量或执行类级别的预处理操作。

多个静态代码块按定义顺序执行，且先于非静态代码块和构造方法。

```
public class Database {
    private static String url;

    // 静态代码块：初始化静态变量
    static {
        url = "jdbc:mysql://localhost:3306/test";
        System.out.println("数据库连接地址初始化完成");
    }
}
```

4、修饰内部类

静态内部类不依赖于外部类的实例，可以独立存在，**不能直接访问外部类的非静态成员**（需通过外部类实例访问）。

当内部类与外部类的实例无关时使用，避免内部类持有外部类的引用导致的内存泄漏。

```
public class OuterClass {
    private static int staticVar = 10;
```

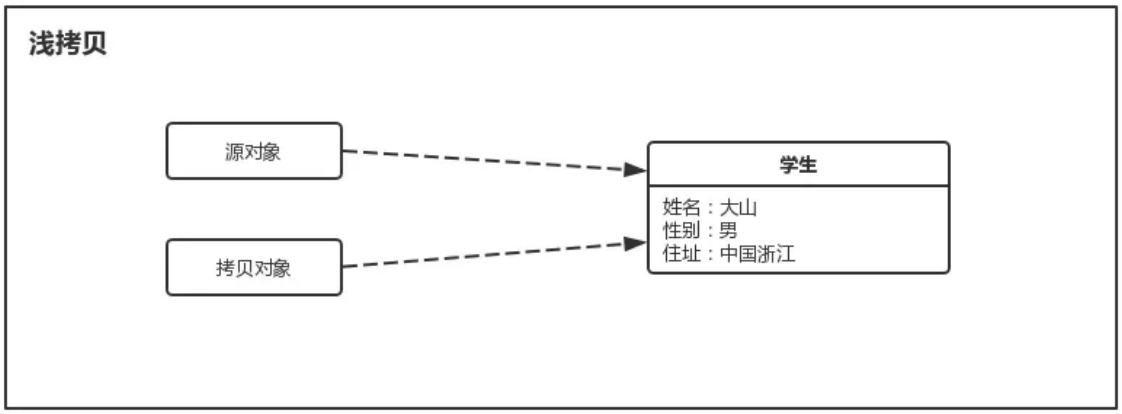
```
    private int instanceVar = 20;

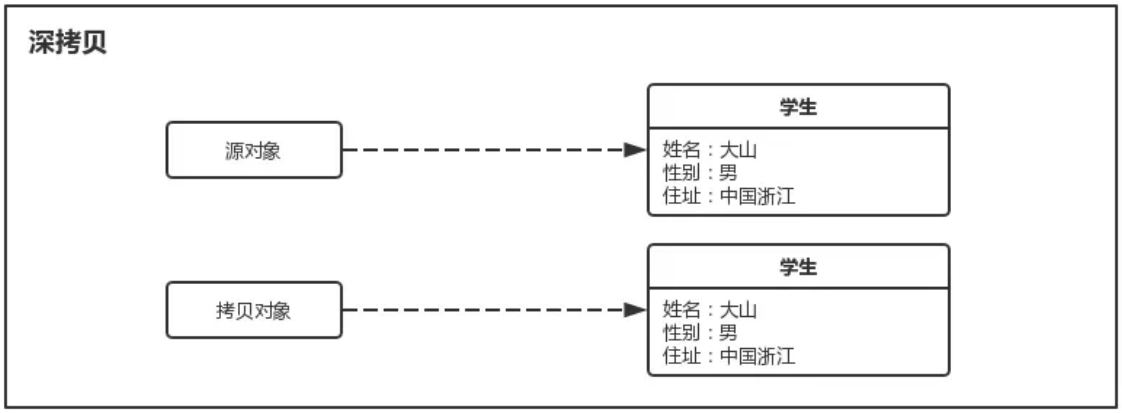
    // 静态内部类
    public static class StaticInnerClass {
        public void print() {
            System.out.println(staticVar); // 可访问外部类静态变量
            // System.out.println(instanceVar); // 错误：不能直接访问非静态变量
        }
    }
}

// 使用静态内部类
public class Test {
    public static void main(String[] args) {
        OuterClass.StaticInnerClass inner = new OuterClass.StaticInnerClass();
        inner.print();
    }
}
```

深拷贝和浅拷贝

深拷贝和浅拷贝的区别?





https://blog.csdn.net/qq_40093255

- 浅拷贝是指只复制对象本身和其内部的值类型字段，但不会复制对象内部的引用类型字段。换句话说，浅拷贝只是创建一个新的对象，然后将原对象的字段值复制到新对象中，但如果原对象内部有引用类型的字段，只是将引用复制到新对象中，两个对象指向的是同一个引用对象。
- 深拷贝是指在复制对象的同时，将对象内部的所有引用类型字段的内容也复制一份，而不是共享引用。换句话说，深拷贝会递归复制对象内部所有引用类型的字段，生成一个全新的对象以及其内部的所有对象。

实现深拷贝的三种方法是什么?

在 Java 中，实现对象深拷贝的方法有以下几种主要方式：

> 实现 Cloneable 接口并重写 clone() 方法

这种方法要求对象及其所有引用类型字段都实现 Cloneable 接口，并且重写 clone() 方法。在 clone() 方法中，通过递归克隆引用类型字段来实现深拷贝。

```
class MyClass implements Cloneable {
    private String field1;
    private NestedClass nestedObject;
```

```
    @Override
    protected Object clone() throws CloneNotSupportedException {
        MyClass cloned = (MyClass) super.clone();
        cloned.nestedObject = (NestedClass) nestedObject.clone(); // 深拷贝内部的引用对象
        return cloned;
    }
}

class NestedClass implements Cloneable {
    private int nestedField;

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}
```

> 使用序列化和反序列化

通过将对象序列化为字节流，再从字节流反序列化为对象来实现深拷贝。要求对象及其所有引用类型字段都实现 Serializable 接口。

```
import java.io.*;

class MyClass implements Serializable {
    private String field1;
    private NestedClass nestedObject;

    public MyClass deepCopy() {
        try {
            ByteArrayOutputStream bos = new ByteArrayOutputStream();
            ObjectOutputStream oos = new ObjectOutputStream(bos);
            oos.writeObject(this);
            oos.flush();
            oos.close();
```

```
            ByteArrayInputStream bis = new ByteArrayInputStream(bos.toByteArray());
            ObjectInputStream ois = new ObjectInputStream(bis);
            return (MyClass) ois.readObject();
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
            return null;
        }
    }
}

class NestedClass implements Serializable {
    private int nestedField;
}
```

> 手动递归复制

针对特定对象结构，手动递归复制对象及其引用类型字段。适用于对象结构复杂度不高的情况。

```
class MyClass {
    private String field1;
    private NestedClass nestedObject;

    public MyClass deepCopy() {
        MyClass copy = new MyClass();
        copy.setField1(this.field1);
        copy.setNestedObject(this.nestedObject.deepCopy());
        return copy;
    }
}

class NestedClass {
    private int nestedField;

    public NestedClass deepCopy() {
        NestedClass copy = new NestedClass();
```

```
        copy.setNestedField(this.nestedField);
        return copy;
    }
}
```

泛型

什么是泛型?

泛型是 Java 编程语言中的一个重要特性，它允许类、接口和方法在定义时使用一个或多个类型参数，这些类型参数在使用时可以被指定为具体的类型。

泛型的主要目的是在编译时提供更强的类型检查，并且在编译后能够保留类型信息，避免了在运行时出现类型转换异常。

> 为什么需要泛型？

- **适用于多种数据类型执行相同的代码**

```
private static int add(int a, int b) {
    System.out.println(a + "+" + b + "=" + (a + b));
    return a + b;
}

private static float add(float a, float b) {
    System.out.println(a + "+" + b + "=" + (a + b));
    return a + b;
}

private static double add(double a, double b) {
    System.out.println(a + "+" + b + "=" + (a + b));
    return a + b;
}
```

如果没有泛型，要实现不同类型的加法，每种类型都需要重载一个 add 方法；通过泛型，我们可以复用为一个方法：

```
private static <T extends Number> double add(T a, T b) {
```

```
    System.out.println(a + "+" + b + "=" + (a.doubleValue() + b.doubleValue()));
    return a.doubleValue() + b.doubleValue();
}
```

- **泛型中的类型在使用时指定，不需要强制类型转换**（**类型安全**，编译器会**检查类型**）

看下这个例子：

```
List list = new ArrayList();
list.add("xxString");
list.add(100d);
list.add(new Person());
```

我们在使用上述 list 中，list 中的元素都是 Object 类型（无法约束其中的类型），所以在取出集合元素时需要人为的强制类型转化到具体的目标类型，且很容易出现 java.lang.ClassCastException 异常。

引入泛型，它将提供类型的约束，提供编译前的检查：

```
List<String> list = new ArrayList<String>();

// list 中只能放 String，不能放其它类型的元素
```

对象

java 创建对象有哪些方式?

在 Java 中，创建对象的方式有多种，常见的包括：

1、使用 new 关键字：这是最常见、最基础的创建对象方式。通过调用类的构造器来实例化对象。

```
// 定义一个类
public class Person {
    private String name;

    public Person() {} // 默认构造器
    public Person(String name) { // 带参构造器
        this.name = name;
    }
```

```
    public void sayHello() {
        System.out.println("Hello, " + name);
    }
}

// 使用 new 创建对象
public class Main {
    public static void main(String[] args) {
        Person person1 = new Person(); // 调用无参构造
        Person person2 = new Person("Alice"); // 调用有参构造

        person2.sayHello(); // 输出: Hello, Alice
    }
}
```

优点是简单、直接、明确。缺点是紧密耦合，必须知道具体的类名。

2、使用 Class 类的 newInstance()方法：通过 Java 的反射 API，在运行时动态地创建对象。这种方式不需要在编译时知道具体的类。

```
注意：Class.newInstance() 在 JDK 9 后被标记为过时，因为它只能调用无参公有构造器，且会抛出所有异常。
Constructor.newInstance() 更强大、更灵活。MyClass obj = (MyClass)
Class.forName("com.example.MyClass").newInstance();
```

应用场景：框架设计（如 Spring 的 IOC 容器）、动态代理等。

注意：`Class.newInstance()` 在 JDK 9 后被标记为过时，因为它只能调用无参公有构造器，且会抛出所有异常。`Constructor.newInstance()` 更强大、更灵活。

使用 Constructor 类的 newInstance()方法：同样是通过反射机制，可以使用 Constructor 类的 newInstance()方法创建对象。

```
Constructor<MyClass> constructor = MyClass.class.getConstructor();
MyClass obj = constructor.newInstance();
```

3、使用 clone()方法：通过实现 `Cloneable` 接口并重写 `Object` 类的 `clone()` 方法，可以基于一个现有对象（原型）创建一个新的副本对象。

```
// 实现 Cloneable 接口
public class Person implements Cloneable {
    private String name;

    // ... 构造器和其他方法 ...

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone(); // 浅拷贝
    }
}

public class Main {
    public static void main(String[] args) {
        Person original = new Person("Charlie");
        try {
            Person copy = (Person) original.clone(); // 创建副本
            copy.sayHello(); // 输出: Hello, Charlie
        } catch (CloneNotSupportedException e) {
            e.printStackTrace();
        }
    }
}
```

`Object.clone()` 默认是浅拷贝，对于引用类型的字段，复制的是引用地址，而不是引用的对象本身。如果需要深拷贝，必须在 `clone()` 方法中手动对引用对象进行克隆。

4、使用反序列化：通过 `ObjectInputStream` 从一个字节流（通常是文件或网络）中重建一个对象。

```
import java.io.*;

// 必须实现 Serializable 接口
public class Person implements Serializable {
    private String name;
    // ... 构造器和其他方法 ...
}
```

```
public class Main {
    public static void main(String[] args) {
        Person personToSave = new Person("David");

        // 序列化对象到文件
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("person.dat"))) {
            oos.writeObject(personToSave);
        } catch (IOException e) {
            e.printStackTrace();
        }

        // 从文件反序列化对象
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("person.dat")))
{
            Person restoredPerson = (Person) ois.readObject(); // 创建新对象
            restoredPerson.sayHello(); // 输出: Hello, David
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
    }
}
```

特点是不会调用类的任何构造器，类必须实现 `java.io.Serializable` 接口。

5、使用工厂模式：这是一种设计模式，不直接使用 `new`，而是通过一个方法来返回对象实例。`getInstance()`、`valueOf()` 等都是常见的工厂方法。

```
public class Person {
    private String name;

    private Person(String name) { // 构造器可以是私有的
        this.name = name;
    }

    // 静态工厂方法
```

```
    public static Person createPerson(String name) {
        // 这里可以做一些额外的逻辑，比如缓存、日志、返回子类实例等
        return new Person(name);
    }
}

public class Main {
    public static void main(String[] args) {
        // 不是用 new，而是用工厂方法创建
        Person person = Person.createPerson("Eva");
        person.sayHello();
    }
}
```

优点是将对象的创建与使用分离，降低耦合，还可以隐藏创建对象的复杂逻辑（如池化技术、缓存）。

Java 标准库中的例子：`Integer.valueOf(int)`，`Calendar.getInstance()`。

最后来个，总结对比：

方式	核心原理	是否调用构造器？	特点与应用场景
`new` 关键字	JVM 指令	**是**	最标准、最常用，紧密耦合
反射	运行时类信息	**是**（`Constructor`）	灵活，解耦，用于框架
`clone()`	复制现有对象	**否**	基于原型创建副本，需实现 `Cloneable`

方式	核心原理	是否调用构造器？	特点与应用场景
反序列化	从字节流恢复	**否**	用于持久化和网络通信，需实现 `Serializable`
工厂模式	方法封装 `new`	**是**（在方法内）	解耦，隐藏创建逻辑，控制实例

Java 创建对象除了 new 还有别的什么方式?

- **通过反射创建对象**：通过 Java 的反射机制可以在运行时动态地创建对象。可以使用 Class 类的 newInstance() 方法或者通过 Constructor 类来创建对象。

```
public class MyClass {
    public MyClass() {
        // Constructor
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        Class<?> clazz = MyClass.class;
        MyClass obj = (MyClass) clazz.newInstance();
    }
}
```

- **通过反序列化创建对象**：通过将对象序列化（保存到文件或网络传输）然后再反序列化（从文件或网络传输中读取对象）的方式来创建对象，对象能被序列化和反序列化的前提是类实现 Serializable 接口。

```
import java.io.*;

public class MyClass implements Serializable {
```

```
    // Class definition
}

public class Main {
    public static void main(String[] args) throws Exception {
        // Serialize object
        MyClass obj = new MyClass();
        ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("object.ser"));
        out.writeObject(obj);
        out.close();

        // Deserialize object
        ObjectInputStream in = new ObjectInputStream(new FileInputStream("object.ser"));
        MyClass newObj = (MyClass) in.readObject();
        in.close();
    }
}
```

- **通过 clone 创建对象**：所有 Java 对象都继承自 Object 类，Object 类中有一个 clone() 方法，可以用来创建对象的副本，要使用 clone 方法，我们必须先实现 Cloneable 接口并实现其定义的 clone 方法。

```
public class MyClass implements Cloneable {
    @Override
    public Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}

public class Main {
    public static void main(String[] args) throws CloneNotSupportedException {
        MyClass obj1 = new MyClass();
        MyClass obj2 = (MyClass) obj1.clone();
    }
}
```

New 出的对象什么时候回收?

通过过关键字`new`创建的对象，由 Java 的垃圾回收器（Garbage Collector）负责回收。垃圾回收器的工作是在程序运行过程中自动进行的，它会周期性地检测不再被引用的对象，并将其回收释放内存。

具体来说，Java 对象的回收时机是由垃圾回收器根据一些算法来决定的，主要有以下几种情况：

1. 引用计数法：某个对象的引用计数为 0 时，表示该对象不再被引用，可以被回收。
2. 可达性分析算法：从根对象（如方法区中的类静态属性、方法中的局部变量等）出发，通过对象之间的引用链进行遍历，如果存在一条引用链到达某个对象，则说明该对象是可达的，反之不可达，不可达的对象将被回收。
3. 终结器（Finalizer）：如果对象重写了`finalize()`方法，垃圾回收器会在回收该对象之前调用`finalize()`方法，对象可以在`finalize()`方法中进行一些清理操作。然而，终结器机制的使用不被推荐，因为它的执行时间是不确定的，可能会导致不可预测的性能问题。

如何获取私有对象?

在 Java 中，私有对象通常指的是类中被声明为`private`的成员变量或方法。由于`private`访问修饰符的限制，这些成员只能在其所在的类内部被访问。

不过，可以通过下面两种方式来间接获取私有对象。

- 使用公共访问器方法（getter 方法）：如果类的设计者遵循良好的编程规范，通常会为私有成员变量提供公共的访问器方法（即`getter`方法），通过调用这些方法可以安全地获取私有对象。

```
class MyClass {
    // 私有成员变量
    private String privateField = "私有字段的值";

    // 公共的 getter 方法
    public String getPrivateField() {
        return privateField;
    }
}

public class Main {
```

```
    public static void main(String[] args) {
        MyClass obj = new MyClass();
        // 通过调用 getter 方法获取私有对象
        String value = obj.getPrivateField();
        System.out.println(value);
    }
}
```

- 反射机制。反射机制允许在运行时检查和修改类、方法、字段等信息，通过反射可以绕过 `private` 访问修饰符的限制来获取私有对象。

```
import java.lang.reflect.Field;

class MyClass {
    private String privateField = "私有字段的值";
}

public class Main {
    public static void main(String[] args) throws NoSuchFieldException, IllegalAccessException
{
        MyClass obj = new MyClass();
        // 获取 Class 对象
        Class<?> clazz = obj.getClass();
        // 获取私有字段
        Field privateField = clazz.getDeclaredField("privateField");
        // 设置可访问性
        privateField.setAccessible(true);
        // 获取私有字段的值
        String value = (String) privateField.get(obj);
        System.out.println(value);
    }
}
```

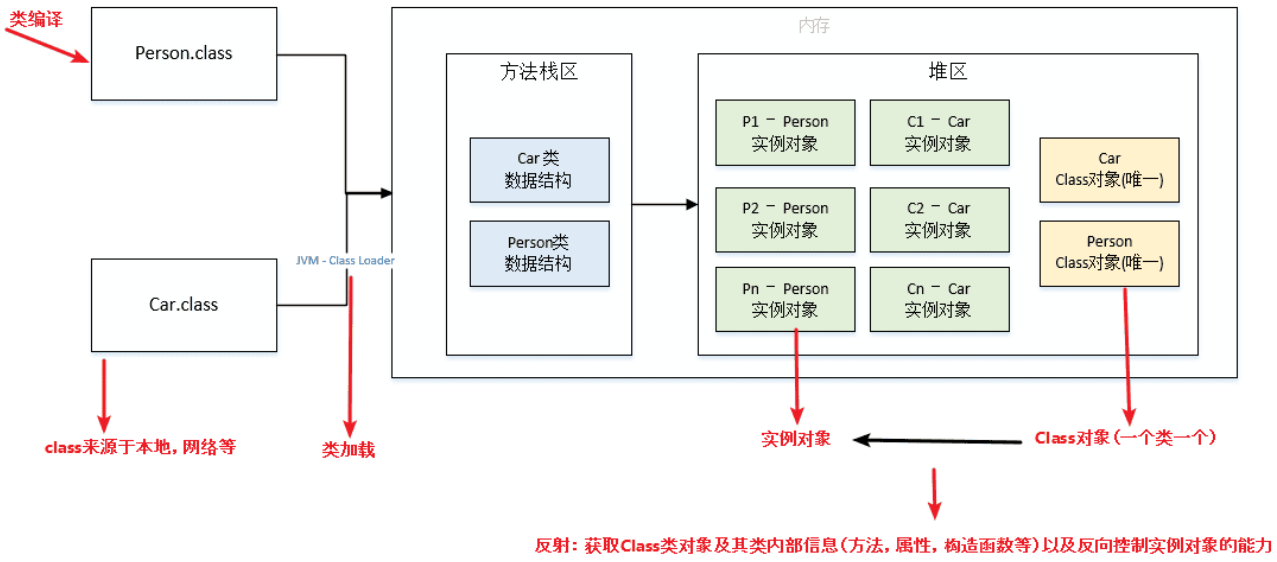
反射

什么是反射?

Java 反射机制是在运行状态中，对于任意一个类，都能够知道这个类中的所有属性和方法，对于任意一个对象，都能够调用它的任意一个方法和属性；这种动态获取的信息以及动态调用对象的方法的功能称为 Java 语言的反射机制。

反射具有以下特性：

1. **运行时类信息访问**：反射机制允许程序在运行时获取类的完整结构信息，包括类名、包名、父类、实现的接口、构造函数、方法和字段等。
2. **动态对象创建**：可以使用反射 API 动态地创建对象实例，即使在编译时不知道具体的类名。这是通过 Class 类的 newInstance()方法或 Constructor 对象的 newInstance()方法实现的。
3. **动态方法调用**：可以在运行时动态地调用对象的方法，包括私有方法。这通过 Method 类的 invoke()方法实现，允许你传入对象实例和参数值来执行方法。
4. **访问和修改字段值**：反射还允许程序在运行时访问和修改对象的字段值，即使是私有的。这是通过 Field 类的 get()和 set()方法完成的。



反射在你平时写代码或者框架中的应用场景有哪些?

> 加载数据库驱动

我们的项目底层数据库有时是用 mysql，有时用 oracle，需要动态地根据实际情况加载驱动类，这个时候反射就有用了，假设 com.mikechen.java.myqlConnection，com.mikechen.java.oracleConnection 这两个类我们要用。

这时候我们在使用 JDBC 连接数据库时使用 Class.forName()通过反射加载数据库的驱动程序，如果是 mysql 则传入 mysql 的驱动类，而如果是 oracle 则传入的参数就变成另一个了。

```
//  DriverManager.registerDriver(new com.mysql.cj.jdbc.Driver());
Class.forName("com.mysql.cj.jdbc.Driver");
```

> 配置文件加载

Spring 框架的 IOC（动态加载管理 Bean），Spring 通过配置文件配置各种各样的 bean，你需要用到哪些 bean 就配哪些，spring 容器就会根据你的需求去动态加载，你的程序就能健壮地运行。

Spring 通过 XML 配置模式装载 Bean 的过程：

- 将程序中所有 XML 或 properties 配置文件加载入内存
- Java 类里面解析 xml 或者 properties 里面的内容，得到对应实体类的字节码字符串以及相关的属性信息
- 使用反射机制，根据这个字符串获得某个类的 Class 实例
- 动态配置实例的属性

配置文件

```
className=com.example.reflectdemo.TestInvoke
methodName=printlnState
```

实体类

```
public class TestInvoke {
    private void printlnState(){
        System.out.println("I am fine");
    }
}
```

解析配置文件内容

```
// 解析 xml 或 properties 里面的内容，得到对应实体类的字节码字符串以及属性信息
public static String getName(String key) throws IOException {
    Properties properties = new Properties();
    FileInputStream in = new FileInputStream("D:\IdeaProjects\AllDemos\language-
specification\src\main\resources\application.properties");
    properties.load(in);
    in.close();
```

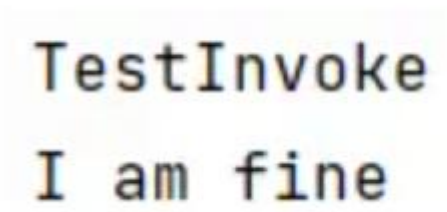
```
    return properties.getProperty(key);
}
```

利用反射获取实体类的 Class 实例，创建实体类的实例对象，调用方法

```
public static void main(String[] args) throws NoSuchMethodException, InvocationTargetException,
IllegalAccessException, IOException, ClassNotFoundException, InstantiationException {
    // 使用反射机制，根据这个字符串获得 Class 对象
    Class<?> c = Class.forName(getName("className"));
    System.out.println(c.getSimpleName());
    // 获取方法
    Method method = c.getDeclaredMethod(getName("methodName"));
    // 绕过安全检查
    method.setAccessible(true);
    // 创建实例对象
    TestInvoke testInvoke = (TestInvoke)c.newInstance();
    // 调用方法
    method.invoke(testInvoke);

}
```

运行结果：



注解

能讲一讲 Java 注解的原理吗?

注解本质是一个继承了 Annotation 的特殊接口，其具体实现类是 Java 运行时生成的动态代理类。

我们通过反射获取注解时，返回的是 Java 运行时生成的动态代理对象。通过代理对象调用自定义注解的方法，会最终调用 AnnotationInvocationHandler 的 invoke 方法。该方法会从 memberValues 这个 Map 中索引出对应的值。而 memberValues 的来源是 Java 常量池。

对注解解析的底层实现了解吗?

注解本质上是一种特殊的接口，它继承自 `java.lang.annotation.Annotation` 接口，**所以注解也叫声明式接口**，例如，定义一个简单的注解：

```
public @interface MyAnnotation {
    String value();
}
```

编译后，Java 编译器会将其转换为一个继承自 `Annotation` 的接口，并生成相应的字节码文件。

根据注解的作用范围，Java 注解可以分为以下几种类型：

- **源码级别注解** ：仅存在于源码中，编译后不会保留（`@Retention(RetentionPolicy.SOURCE)`）。
- **类文件级别注解** ：保留在 `.class` 文件中，但运行时不可见（`@Retention(RetentionPolicy.CLASS)`）。
- **运行时注解** ：保留在 `.class` 文件中，并且可以通过反射在运行时访问（`@Retention(RetentionPolicy.RUNTIME)`）。

只有运行时注解可以通过反射机制进行解析。

当注解被标记为 `RUNTIME` 时，Java 编译器会在生成的 `.class` 文件中保存注解信息。这些信息存储在字节码的属性表（Attribute Table）中，具体包括以下内容：

- RuntimeVisibleAnnotations ：存储运行时可见的注解信息。
- RuntimeInvisibleAnnotations ：存储运行时不可见的注解信息。
- RuntimeVisibleParameterAnnotations 和 RuntimeInvisibleParameterAnnotations ：存储方法参数上的注解信息。

通过工具（如 `javap -v`）可以查看 `.class` 文件中的注解信息。

注解的解析主要依赖于 Java 的反射机制。以下是解析注解的基本流程：

1、获取注册信息：通过反射 API 可以获取类、方法、字段等元素上的注解。例如：

```
Class<?> clazz = MyClass.class;
MyAnnotation annotation = clazz.getAnnotation(MyAnnotation.class);
if (annotation != null) {
    System.out.println(annotation.value());
```

```
}
```

2、底层原理：反射机制的核心类是 `java.lang.reflect.AnnotatedElement`，它是所有可以被注解修饰的元素（如 `Class`、`Method`、`Field` 等）的父接口。该接口提供了以下方法：

- `getAnnotation(Class<T> annotationClass)`：获取指定类型的注解。
- `getAnnotations()`：获取所有注解。
- `isAnnotationPresent(Class<? extends Annotation> annotationClass)`：判断是否包含指定注解。

这些方法的底层实现依赖于 JVM 提供的本地方法（Native Method），例如：

- `native Annotation[] getDeclaredAnnotations0(boolean publicOnly);`
- `native <A extends Annotation> A getAnnotation(Class<A> annotationClass);`

JVM 在加载类时会解析 `.class` 文件中的注解信息，并将其存储在内存中，供反射机制使用。

因此，注解解析的底层实现主要依赖于 Java 的反射机制和字节码文件的存储。通过 `@Retention` 元注解可以控制注解的保留策略，当使用 `RetentionPolicy.RUNTIME` 时，可以在运行时通过反射 API 来解析注解信息。在 JVM 层面，会从字节码文件中读取注解信息，并创建注解的代理对象来获取注解的属性值。

Java 注解的作用域呢?

注解的作用域（Scope）指的是注解可以应用在哪些程序元素上，例如类、方法、字段等。Java 注解的作用域可以分为三种：

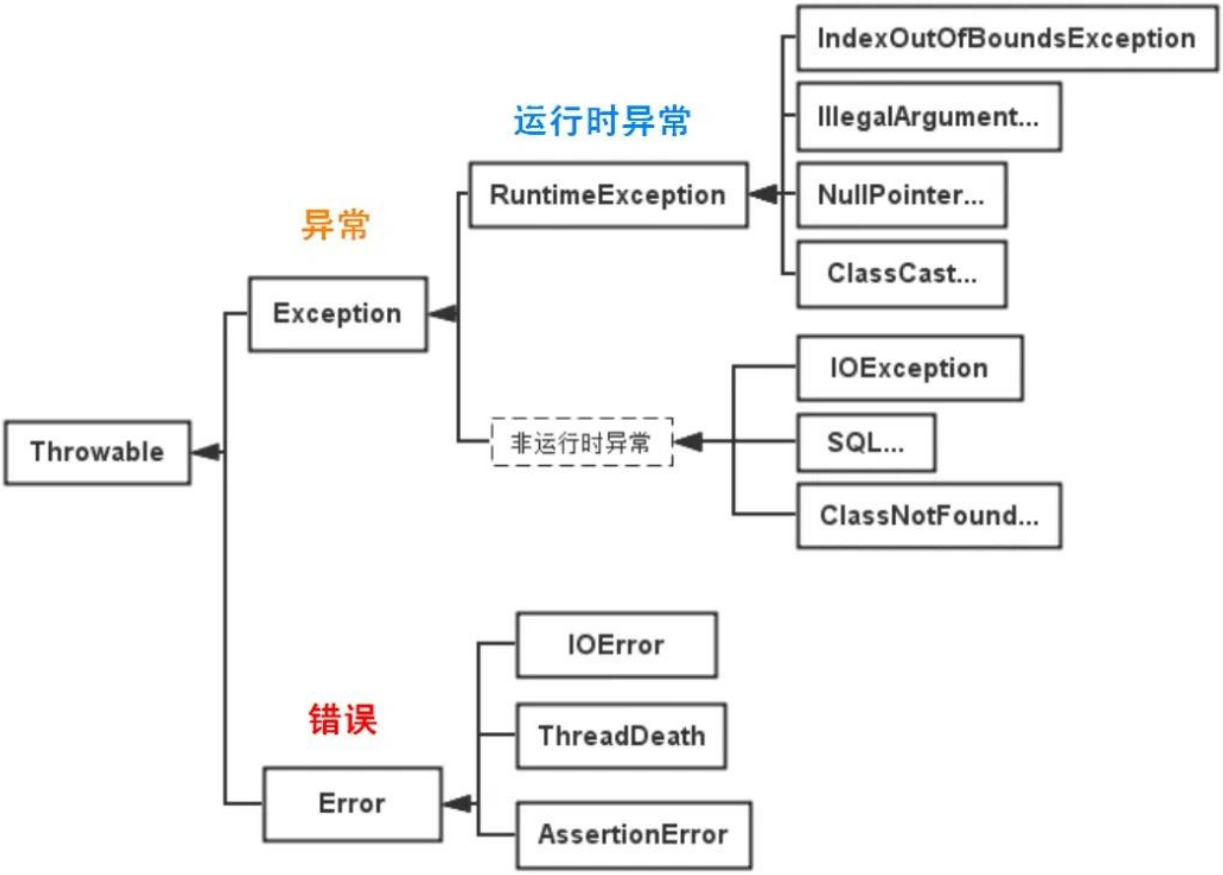
1. 类级别作用域：用于描述类的注解，通常放置在类定义的上面，可以用来指定类的一些属性，如类的访问级别、继承关系、注释等。
2. 方法级别作用域：用于描述方法的注解，通常放置在方法定义的上面，可以用来指定方法的一些属性，如方法的访问级别、返回值类型、异常类型、注释等。
3. 字段级别作用域：用于描述字段的注解，通常放置在字段定义的上面，可以用来指定字段的一些属性，如字段的访问级别、默认值、注释等。

除了这三种作用域，Java 还提供了其他一些注解作用域，例如构造函数作用域和局部变量作用域。这些注解作用域可以用来对构造函数和局部变量进行描述和注释。

异常

介绍一下 Java 异常

Java 异常类层次结构图：



Java 的异常体系主要基于 `Throwable` 及其子类。`Throwable` 有两个重要的子类：`Error` 和 `Exception`，它们分别代表了不同类型的异常情况。

1. **`Error`（错误）：** 表示运行环境的错误，错误是程序无法处理的严重问题，如虚拟机错误、动态链接库失效等。程序不应该尝试捕获这类错误。例如，`OutOfMemoryError`、`StackOverflowError` 等。
2. **`Exception`（异常）：** 表示程序本身可以处理的异常情况。异常分为两大类：
    - **非运行时异常（受检异常，Checked Exception）：** 这类异常在编译时就必须被捕获或者声明抛出。它们通常是外部错误，如文件不存在（`FileNotFoundException`）、类未找到（`ClassNotFoundException`）等。非运行时异常强制程序员处理这些可能出现的问题，增强了程序的健壮性。
    - **运行时异常（非受检异常，Unchecked Exception 或 RuntimeException）：** 这类异常特指 `RuntimeException` 及其子类。它与 `Error` 一起构成了 Java 中的**非受检异常**家族。运行时异常由程序逻辑错误导致，如空指针访问（`NullPointerException`）、数组越界（`ArrayIndexOutOfBoundsException`）等。运行时异常是不需要在编译时强制捕获或声明的。

Java 异常处理有哪些?

异常处理是通过使用 try-catch 语句块来捕获和处理异常。以下是 Java 中常用的异常处理方式：

- try-catch 语句块：用于捕获并处理可能抛出的异常。try 块中包含可能抛出异常的代码，catch 块用于捕获并处理特定类型的异常。可以有多个 catch 块来处理不同类型的异常。

```
try {
    // 可能抛出异常的代码
} catch (ExceptionType1 e1) {
    // 处理异常类型 1 的逻辑
} catch (ExceptionType2 e2) {
    // 处理异常类型 2 的逻辑
} catch (ExceptionType3 e3) {
    // 处理异常类型 3 的逻辑
} finally {
    // 可选的 finally 块，用于定义无论是否发生异常都会执行的代码
}
```

- throw 语句：用于手动抛出异常。可以根据需要在代码中使用 throw 语句主动抛出特定类型的异常。

```
throw new ExceptionType("Exception message");
```

- throws 关键字：用于在方法声明中声明可能抛出的异常类型。如果一个方法可能抛出异常，但不想在方法内部进行处理，可以使用 throws 关键字将异常传递给调用者来处理。

```
public void methodName() throws ExceptionType {
    // 方法体
}
```

- finally 块：用于定义无论是否发生异常都会执行的代码块。通常用于释放资源，确保资源的正确关闭。

```
try {
    // 可能抛出异常的代码
} catch (ExceptionType e) {
    // 处理异常的逻辑
```

```
} finally {
    // 无论是否发生异常，都会执行的代码
}
```

抛出异常为什么不用 throws?

如果异常是未检查异常或者在方法内部被捕获和处理了，那么就不需要使用 throws。

- Unchecked Exceptions：未检查异常（unchecked exceptions）是继承自 RuntimeException 类或 Error 类的异常，编译器不强制要求进行异常处理。因此，对于这些异常，不需要在方法签名中使用 throws 来声明。示例包括 NullPointerException、ArrayIndexOutOfBoundsException 等。
- **捕获和处理异常**：另一种常见情况是，在方法内部捕获了可能抛出的异常，并在方法内部处理它们，而不是通过 throws 子句将它们传递到调用者。这种情况下，方法可以处理异常而无需在方法签名中使用 throws。

try catch 中的语句运行情况

try 块中的代码将按顺序执行，如果抛出异常，将在 catch 块中进行匹配和处理，然后程序将继续执行 catch 块之后的代码。如果没有匹配的 catch 块，异常将被传递给上一层调用的方法。

try{return “a”} finally{return “b”}这条语句返回啥

finally 块中的 return 语句会覆盖 try 块中的 return 返回，因此，该语句将返回"b"。

object

Object 类有哪些方法?

Java Object 类是所有类的超类，默认提供 11 个核心方法，核心用于对象比较、哈希、字符串表示、线程同步等。

首先最常用的是 equals 方法，它的默认实现是比较两个对象的内存地址，也就是和 == 的效果一样，但实际开发中我们常需要按对象的内容比较，比如两个用户对象只要 id 相同就认为相等，这时候就需要重写 equals。比如：

```
class User {
    private int id;
    private String name;
```

```
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        User user = (User) obj;
        return id == user.id;
    }
}
```

和 equals 配套的必须重写 hashCode 方法，因为 Java 的约定是如果两个对象 equals 返回 true，它们的 hashCode 必须相等；如果 hashCode 不相等，equals 一定返回 false。如果只重写 equals 不重写 hashCode，会导致对象在 HashMap HashSet 等集合中无法正确存储，比如两个 id 相同的 User 对象，equals 返回 true，但 hashCode 不同，会被当成两个不同元素存入集合。重写示例：

```
@Override
public int hashCode() {
    return Integer.hashCode(id);
}
```

然后是 toString 方法，默认返回类名加 @加对象的哈希码十六进制表示，比如 User@1b6d3586，可读性很差。实际开发中重写它是为了返回对象的具体信息，方便日志打印和调试，比如：

```
@Override
public String toString() {
    return User  id: + id +  name: + name;
}
```

接下来是 getClass 方法，它返回对象运行时的实际类对象，和编译时类型可能不同，而且这个方法不能重写。比如父类 Animal 的子类 Dog，Animal animal = new Dog，animal.getClass 返回的是 Dog 的类对象，常用于反射场景，比如通过 getClass 获取类的属性和方法：

```
Animal animal = new Dog();
Class<?> clazz = animal.getClass();
System.out.println(clazz.getName()); // 输出 Dog
```

然后是 clone 方法，用于创建对象的浅拷贝，默认是浅拷贝意味着如果对象有引用类型属性，拷贝后的对象和原对象会共享该引用属性。使用 clone 需要让类实现 Cloneable 接口，否则会抛出 CloneNotSupportedException，比如：

```
class Product implements Cloneable {
    private String name;
    private double price;

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}
```

还有 notify 和 notifyAll 方法，它们都是用于多线程同步的，和 synchronized 配合使用，作用是唤醒等待当前对象锁的线程。notify 是随机唤醒一个等待线程，notifyAll 是唤醒所有等待线程，比如生产者消费者模式中，生产者生产完数据后调用 notifyAll，唤醒等待的消费者线程。

wait 方法有三个重载，作用是让当前持有对象锁的线程释放锁并进入等待状态，直到被 notify notifyAll 唤醒或者等待时间到期。同样需要在 synchronized 同步块或方法中使用，否则会抛 IllegalMonitorStateException，比如：

```
synchronized (lockObj) {
    while (条件不满足) {
        lockObj.wait(1000); // 等待 1 秒，超时自动唤醒
    }
    // 执行业务逻辑
}
```

最后是 finalize 方法，它是对象被垃圾回收器回收前会调用的方法，默认是空实现。但现在基本不推荐使用，因为它的执行时机不确定，可能很久才执行甚至不执行，而且可能导致对象复活，影响垃圾回收效率，Java9 之后已经标记为过时，替代方案是使用 try with resources 或者 PhantomReference 来处理资源释放。

== 与 equals 有什么区别?

简单来说，`==` **比较的是"地址"，而** `equals` **比较的是"内容"**。但这句话如果不展开讲，其实很容易让人懵，我给你详细说一下。

先说 `==` **这个运算符。**

`==` 在 Java 里其实是个非常**简单粗暴**的比较方式。如果你比较的是**基本数据类型**（比如 int、double），那它就直接比数值是不是相等，这个没啥问题。

但如果你比较的是**对象（引用类型）**，那 `==` 比较的就不是对象里面装的内容了，而是比较这两个变量**是不是指向内存中同一个对象**，也就是**地址是否相同**。

我举个例子你就明白了：

```
String a = new String("hello");
String b = new String("hello");

System.out.println(a == b);  // 输出 false
```

虽然 a 和 b 里面装的内容都是 "hello"，但因为我用了两次 `new`，所以在堆内存里创建了两个完全独立的 String 对象。a 和 b 这两个变量分别指向了不同的对象，所以 `==` 一比较，发现地址不一样，就返回 false 了。

再说 `equals` **方法。**

`equals` 是 Object 类里定义的一个方法，所有的 Java 对象都继承了它。它的**默认行为其实和** `==` **一模一样**，也是比地址。

但关键在于，很多常用的类（比如 String、Integer）都把这个方法给**重写**了。重写之后，`equals` 就不再比地址了，而是去比较**对象里面实际存储的内容**是否相等。

还是刚才那个例子：

```
String a = new String("hello");
String b = new String("hello");

System.out.println(a.equals(b));  // 输出 true
```

虽然 a 和 b 是两个不同的对象，但 String 类重写了 `equals` 方法，会逐个字符去比较内容。发现都是 "hello"，所以返回 true。

这里有个经典的面试陷阱，就是字符串常量池的问题。

```
String c = "hello";
String d = "hello";


System.out.println(c == d);  // 输出 true
```

这里为什么 `==` 比较也是 true 呢？因为当你直接用双引号创建字符串的时候，JVM 会把它扔到一个叫"字符串常量池"的地方。如果池子里已经有了 "hello"，那 d 就直接复用 c 指向的那个对象，所以它俩地址是一样的，`==` 自然就返回 true 了。

总结一下：

- `==` 是比较**引用地址**，适合用来判断两个变量是不是指向同一个对象；
- `equals` 是比较**对象内容**，但前提是这个类重写了 equals 方法（比如 String），如果没重写，那它俩效果是一样的。

所以在实际开发中，如果我们要比较两个字符串或者对象的内容是否相等，一定要用 `equals`，而不是 `==`。这也是为什么阿里巴巴开发规范里明确要求，判断字符串相等必须用 `equals` 的原因。

hashcode 和 equals 方法有什么关系?

在 Java 中，对于重写 `equals` 方法的类，通常也需要重写 `hashCode` 方法，并且需要遵循以下规定：

- **一致性**：如果两个对象使用 `equals` 方法比较结果为 `true`，那么它们的 `hashCode` 值必须相同。也就是说，如果 `obj1.equals(obj2)` 返回 `true`，那么 `obj1.hashCode()` 必须等于 `obj2.hashCode()`。
- **非一致性**：如果两个对象的 `hashCode` 值相同，它们使用 `equals` 方法比较的结果不一定为 `true`。即 `obj1.hashCode() == obj2.hashCode()` 时，`obj1.equals(obj2)` 可能为 `false`，这种情况称为哈希冲突。

`hashCode` 和 `equals` 方法是紧密相关的，重写 `equals` 方法时必须重写 `hashCode` 方法，以保证在使用哈希表等数据结构时，对象的相等性判断和存储查找操作能够正常工作。而重写 `hashCode` 方法时，需要确保相等的对象具有相同的哈希码，但相同哈希码的对象不一定相等。

java 里 string 的常用方法有哪些?

我常用的 String 常用方法有这些：

- **获取长度**：`int length()`：返回字符串的长度（字符个数）。例：`"abc".length()` → 3

- **判断内容**：`boolean equals(Object obj)`：比较两个字符串的内容是否完全相同（区分大小写）。例：`"abc".equals("ABC")` → false
- **截取子串**：`String substring(int beginIndex)`：从指定索引开始截取到末尾。例：`"hello".substring(2)` → "llo"
- **去除空格**：`String trim()`：去除字符串首尾的空白字符（空格、制表符等）。例：`" abc ".trim()` → "abc"
- **替换内容**：`String replace(char oldChar, char newChar)`：替换所有指定字符。例：`"aaa".replace('a', 'b')` → "bbb"
- **判断空字符串**：`boolean isEmpty()`：判断字符串长度是否为 0（注意：`null` 调用会报错，需先判空）。

String、StringBuffer、StringBuilder 的区别和联系

1、可变性 ：`String` 是不可变的（Immutable），一旦创建，内容无法修改，每次修改都会生成一个新的对象。`StringBuilder` 和 `StringBuffer` 是可变的（Mutable），可以直接对字符串内容进行修改而不会创建新对象。

2、线程安全性 ：`String` 因为不可变，天然线程安全。`StringBuilder` 不是线程安全的，适用于单线程环境。`StringBuffer` 是线程安全的，其方法通过 `synchronized` 关键字实现同步，适用于多线程环境。

3、性能 ：`String` 性能最低，尤其是在频繁修改字符串时会生成大量临时对象，增加内存开销和垃圾回收压力。`StringBuilder` 性能最高，因为它没有线程安全的开销，适合单线程下的字符串操作。`StringBuffer` 性能略低于 `StringBuilder`，因为它的线程安全机制引入了同步开销。

4、使用场景 ：如果字符串内容固定或不常变化，优先使用 `String`。如果需要频繁修改字符串且在单线程环境下，使用 `StringBuilder`。如果需要频繁修改字符串且在多线程环境下，使用 `StringBuffer`。

对比总结如下：

特性	String	StringBuilder	StringBuffer
不可变性	不可变	可变	可变

特性	String	StringBuilder	StringBuffer
线程安全	是（因不可变）	否	是（同步方法）
性能	低（频繁修改时）	高（单线程）	中（多线程安全）
适用场景	静态字符串	单线程动态字符串	多线程动态字符串

例子代码如下：

```
// String 的不可变性
String str = "abc";
str = str + "def"; // 新建对象，str 指向新对象

// StringBuilder（单线程高效）
StringBuilder sb = new StringBuilder();
sb.append("abc").append("def"); // 直接修改内部数组

// StringBuffer（多线程安全）
StringBuffer sbf = new StringBuffer();
sbf.append("abc").append("def"); // 同步方法保证线程安全
```

Java 新特性

Java 8 你知道有什么新特性?

下面是 Java 8 主要新特性的整理表格，包含关键改进和示例说明：

特性名称	描述	示例或说明
Lambda 表达式	简化匿名内部类，支持函数式编程	`(a, b) -> a + b` 代替匿名类实现接口

特性名称	描述	示例或说明
函数式接口	仅含一个抽象方法的接口，可用 `@FunctionalInterface` 注解标记	`Runnable`，`Comparator`，或自定义接口 `@FunctionalInterface interface MyFunc { void run(); }`
Stream API	提供链式操作处理集合数据，支持并行处理	`list.stream().filter(x -> x > 0).collect(Collectors.toList())`
Optional 类	封装可能为 `null` 的对象，减少空指针异常	`Optional.ofNullable(value).orElse("default")`
方法引用	简化 Lambda 表达式，直接引用现有方法	`System.out::println` 等价于 `x -> System.out.println(x)`
接口的默认方法与静态方法	接口可定义默认实现和静态方法，增强扩展性	`interface A { default void print() { System.out.println("默认方法"); } }`
并行数组排序	使用多线程加速数组排序	`Arrays.parallelSort(array)`
重复注解	允许同一位置多次使用相同注解	`@Repeatable` 注解配合容器注解使用

特性名称	描述	示例或说明
类型注解	注解可应用于更多位置（如泛型、异常等）	`List<@NonNull String> list`
CompletableFuture	增强异步编程能力，支持链式调用和组合操作	`CompletableFuture.supplyAsync(() -> "result").thenAccept(System.out::println)`

Lambda 表达式了解吗?

Lambda 表达式它是一种简洁的语法，用于创建匿名函数，主要用于简化函数式接口（只有一个抽象方法的接口）的使用。其基本语法有以下两种形式：

- `(parameters) -> expression`：当 Lambda 体只有一个表达式时使用，表达式的结果会作为返回值。
- `(parameters) -> { statements; }`：当 Lambda 体包含多条语句时，需要使用大括号将语句括起来，若有返回值则需要使用 `return` 语句。

传统的匿名内部类实现方式代码较为冗长，而 Lambda 表达式可以用更简洁的语法实现相同的功能。比如，使用匿名内部类实现 `Runnable` 接口

```
public class AnonymousClassExample {
    public static void main(String[] args) {
        Thread t1 = new Thread(new Runnable() {
            @Override
            public void run() {
                System.out.println("Running using anonymous class");
            }
        });
        t1.start();
    }
}
```

使用 Lambda 表达式实现相同功能：

```
public class LambdaExample {
    public static void main(String[] args) {
        Thread t1 = new Thread(() -> System.out.println("Running using lambda expression"));
        t1.start();
    }
}
```

可以看到，Lambda 表达式的代码更加简洁明了。

还有，Lambda 表达式能够更清晰地表达代码的意图，尤其是在处理集合操作时，如过滤、映射等。比如，过滤出列表中所有偶数

```
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class ReadabilityExample {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6);
        // 使用 Lambda 表达式结合 Stream API 过滤偶数
        List<Integer> evenNumbers = numbers.stream()
                                           .filter(n -> n % 2 == 0)
                                           .collect(Collectors.toList());
        System.out.println(evenNumbers);
    }
}
```

通过 Lambda 表达式，代码的逻辑更加直观，易于理解。

还有，Lambda 表达式使得 Java 支持函数式编程范式，允许将函数作为参数传递，从而可以编写更灵活、可复用的代码。比如定义一个通用的计算函数。

```
interface Calculator {
    int calculate(int a, int b);
}
```

```
public class FunctionalProgrammingExample {
    public static int operate(int a, int b, Calculator calculator) {
        return calculator.calculate(a, b);
    }

    public static void main(String[] args) {
        // 使用 Lambda 表达式传递加法函数
        int sum = operate(3, 5, (x, y) -> x + y);
        System.out.println("Sum: " + sum);

        // 使用 Lambda 表达式传递乘法函数
        int product = operate(3, 5, (x, y) -> x * y);
        System.out.println("Product: " + product);
    }
}
```

虽然 Lambda 表达式优点蛮多的，不过也有一些缺点，比如会增加调试困难，因为 Lambda 表达式是匿名的，在调试时很难定位具体是哪个 Lambda 表达式出现了问题。尤其是当 Lambda 表达式嵌套使用或者比较复杂时，调试难度会进一步增加。

Java 中 stream 的 API 介绍一下

Java 8 引入了 Stream API，它提供了一种高效且易于使用的数据处理方式，特别适合集合对象的操作，如过滤、映射、排序等。Stream API 不仅可以提高代码的可读性和简洁性，还能利用多核处理器的优势进行并行处理。让我们通过两个具体的例子来感受下 Java Stream API 带来的便利，对比在 Stream API 引入之前的传统做法。

> 案例 1：过滤并收集满足条件的元素

问题场景：从一个列表中筛选出所有长度大于 3 的字符串，并收集到一个新的列表中。

没有 Stream API 的做法：

```
List<String> originalList = Arrays.asList("apple", "fig", "banana", "kiwi");
List<String> filteredList = new ArrayList<>();

for (String item : originalList) {
    if (item.length() > 3) {
```

```
        filteredList.add(item);
    }
}
```

这段代码需要显式地创建一个新的 ArrayList，并通过循环遍历原列表，手动检查每个元素是否满足条件，然后添加到新列表中。

使用 Stream API 的做法：

```
List<String> originalList = Arrays.asList("apple", "fig", "banana", "kiwi");
List<String> filteredList = originalList.stream()
                                        .filter(s -> s.length() > 3)
                                        .collect(Collectors.toList());
```

这里，我们直接在原始列表上调用`.stream()`方法创建了一个流，使用`.filter()`中间操作筛选出长度大于 3 的字符串，最后使用`.collect(Collectors.toList())`终端操作将结果收集到一个新的列表中。代码更加简洁明了，逻辑一目了然。

> 案例 2：计算列表中所有数字的总和

问题场景：计算一个数字列表中所有元素的总和。

没有 Stream API 的做法：

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
int sum = 0;
for (Integer number : numbers) {
    sum += number;
}
```

这个传统的 for-each 循环遍历列表中的每一个元素，累加它们的值来计算总和。

使用 Stream API 的做法：

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
int sum = numbers.stream()
                 .mapToInt(Integer::intValue)
                 .sum();
```

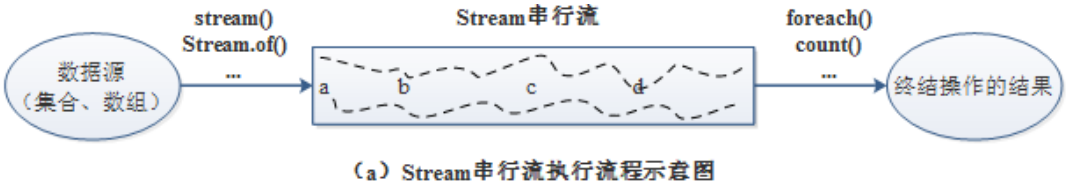
通过 Stream API，我们可以先使用`.mapToInt()`将 Integer 流转换为 IntStream（这是为了高效处理基本类型），然后直接调用`.sum()`方法来计算总和，极大地简化了代码。

Stream 流的并行 API 是什么?

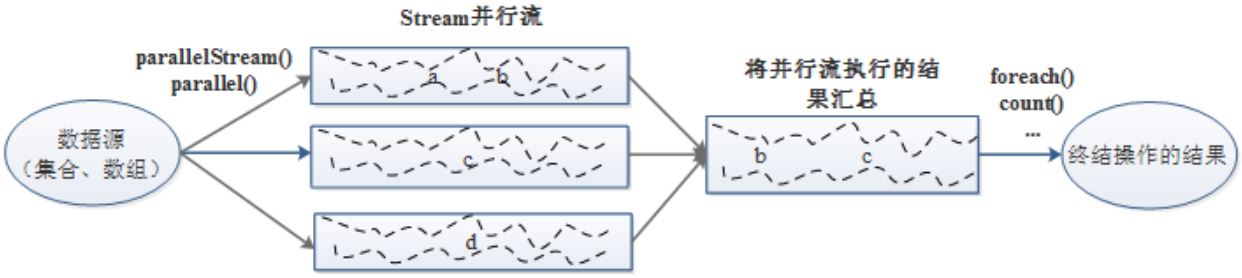
是 ParallelStream。

并行流（ParallelStream）就是将源数据分为多个子流对象进行多线程操作，然后将处理的结果再汇总为一个流对象，底层是使用通用的 fork/join 池来实现，即将一个任务拆分成多个“小任务”并行计算，再把多个“小任务”的结果合并成总的计算结果

Stream 串行流与并行流的主要区别：



（a）Stream串行流执行流程示意图



对 CPU 密集型的任务来说，并行流使用 ForkJoinPool 线程池，为每个 CPU 分配一个任务，这是非常有效率的，但是如果任务不是 CPU 密集的，而是 I/O 密集的，并且任务数相对线程数比较大，那么直接用 ParallelStream 并不是很好的选择。

completableFuture 怎么用的?

CompletableFuture 是由 Java 8 引入的，在 Java8 之前我们一般通过 Future 实现异步。

- Future 用于表示异步计算的结果，只能通过阻塞或者轮询的方式获取结果，而且不支持设置回调方法，Java 8 之前若要设置回调一般会使用 guava 的 ListenableFuture，回调的引入又会导致臭名昭著的回调地狱（下面的例子会通过 ListenableFuture 的使用来具体进行展示）。
- CompletableFuture 对 Future 进行了扩展，可以通过设置回调的方式处理计算结果，同时也支持组合操作，支持进一步的编排，同时一定程度解决了回调地狱的问题。

下面将举例来说明，我们通过 ListenableFuture、CompletableFuture 来实现异步的差异。假设有三个操作 step1、step2、step3 存在依赖关系，其中 step3 的执行依赖 step1 和 step2 的结果。

Future(ListenableFuture)的实现（回调地狱）如下：

```
ExecutorService executor = Executors.newFixedThreadPool(5);
ListeningExecutorService guavaExecutor = MoreExecutors.listeningDecorator(executor);
ListenableFuture<String> future1 = guavaExecutor.submit(() -> {
    //step 1
    System.out.println("执行 step 1");
    return "step1 result";
});
ListenableFuture<String> future2 = guavaExecutor.submit(() -> {
    //step 2
    System.out.println("执行 step 2");
    return "step2 result";
});
ListenableFuture<List<String>> future1And2 = Futures.allAsList(future1, future2);
Futures.addCallback(future1And2, new FutureCallback<List<String>>() {
    @Override
    public void onSuccess(List<String> result) {
        System.out.println(result);
        ListenableFuture<String> future3 = guavaExecutor.submit(() -> {
            System.out.println("执行 step 3");
            return "step3 result";
        });
        Futures.addCallback(future3, new FutureCallback<String>() {
            @Override
            public void onSuccess(String result) {
                System.out.println(result);
            }
            @Override
            public void onFailure(Throwable t) {
            }
        }, guavaExecutor);
    }

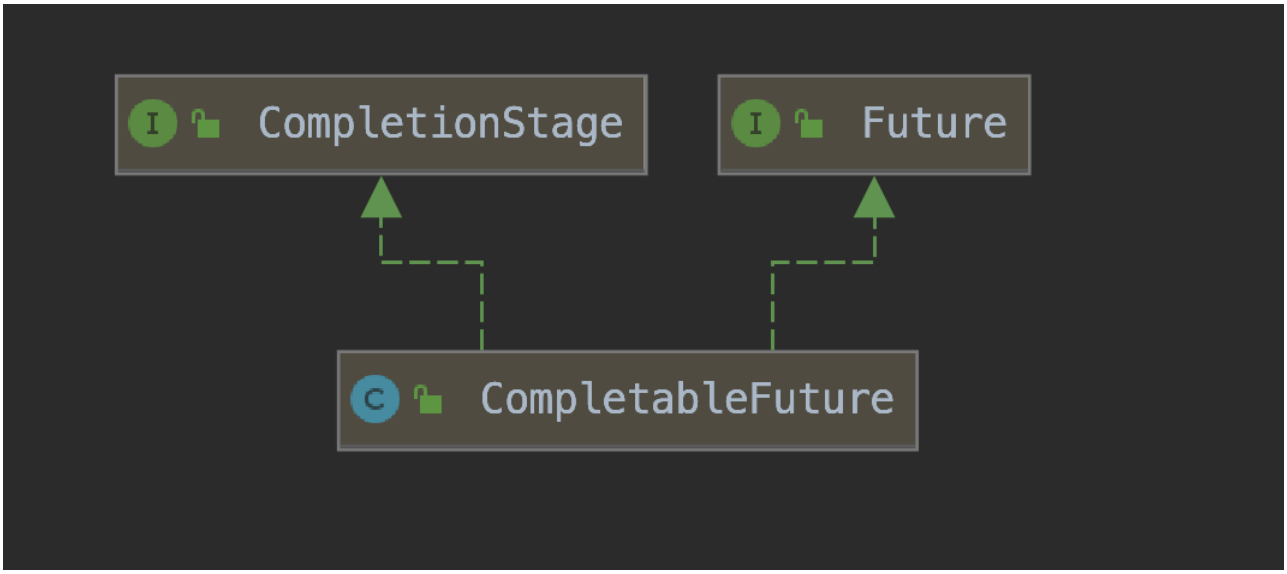
    @Override
```

```
    public void onFailure(Throwable t) {
    }}, guavaExecutor);
```

CompletableFuture 的实现如下：

```
ExecutorService executor = Executors.newFixedThreadPool(5);
CompletableFuture<String> cf1 = CompletableFuture.supplyAsync(() -> {
    System.out.println("执行 step 1");
    return "step1 result";
}, executor);
CompletableFuture<String> cf2 = CompletableFuture.supplyAsync(() -> {
    System.out.println("执行 step 2");
    return "step2 result";
});
cf1.thenCombine(cf2, (result1, result2) -> {
    System.out.println(result1 + " , " + result2);
    System.out.println("执行 step 3");
    return "step3 result";
}).thenAccept(result3 -> System.out.println(result3));
```

显然，CompletableFuture 的实现更为简洁，可读性更好。



CompletableFuture 实现了两个接口（如上图所示）：Future、CompletionStage。

- Future 表示异步计算的结果，CompletionStage 用于表示异步执行过程中的一个步骤（Stage），这个步骤可能是由另外一个 CompletionStage 触发的，随着当前步骤的完成，也可能会触发其他一系列 CompletionStage 的执行。

- 从而我们可以根据实际业务对这些步骤进行多样化的编排组合，CompletionStage 接口正是定义了这样的能力，我们可以通过其提供的 thenAppy、thenCompose 等函数式编程方法来组合编排这些步骤。

Java 21 新特性知道哪些?

新新语言特性：

- **Switch 语句的模式匹配**：该功能在 Java 21 中也得到了增强。它允许在 `switch` 的 `case` 标签中使用模式匹配，使操作更加灵活和类型安全，减少了样板代码和潜在错误。例如，对于不同类型的账户类，可以在 `switch` 语句中直接根据账户类型的模式来获取相应的余额，如 `case savingsAccount sa -> result = sa.getSavings();`
- **数组模式**：将模式匹配扩展到数组中，使开发者能够在条件语句中更高效地解构和检查数组内容。例如，`if (arr instanceof int[] {1, 2, 3})`，可以直接判断数组 `arr` 是否匹配指定的模式。
- **字符串模板（预览版）**：提供了一种更可读、更易维护的方式来构建复杂字符串，支持在字符串字面量中直接嵌入表达式。例如，以前可能需要使用`"hello " + name + ", welcome to the geeksforgeeks!"`这样的方式来拼接字符串，在 Java 21 中可以使用 `hello {name}, welcome to the geeksforgeeks!`这种更简洁的写法

新并发特性方面：

- **虚拟线程**：这是 Java 21 引入的一种轻量级并发的新选择。它通过共享堆栈的方式，大大降低了内存消耗，同时提高了应用程序的吞吐量和响应速度。可以使用静态构建方法、构建器或 `ExecutorService` 来创建和使用虚拟线程。
- Scoped Values（**范围值**）：提供了一种在线程间共享不可变数据的新方式，避免使用传统的线程局部存储，促进了更好的封装性和线程安全，可用于在不通过方法参数传递的情况下，传递上下文信息，如用户会话或配置设置。

序列化

怎么把一个对象从一个 jvm 转移到另一个 jvm?

- **使用序列化和反序列化**：将对象序列化为字节流，并将其发送到另一个 JVM，然后在另一个 JVM 中反序列化字节流恢复对象。这可以通过 Java 的 ObjectOutputStream 和 ObjectInputStream 来实现。

- **使用消息传递机制**：利用消息传递机制，比如使用消息队列（如 RabbitMQ、Kafka）或者通过网络套接字进行通信，将对象从一个 JVM 发送到另一个。这需要自定义协议来序列化对象并在另一个 JVM 中反序列化。
- **使用远程方法调用（RPC）**：可以使用远程方法调用框架，如 gRPC，来实现对象在不同 JVM 之间的传输。远程方法调用可以让你在分布式系统中调用远程 JVM 上的对象的方法。
- **使用共享数据库或缓存**：将对象存储在共享数据库（如 MySQL、PostgreSQL）或共享缓存（如 Redis）中，让不同的 JVM 可以访问这些共享数据。这种方法适用于需要共享数据但不需要直接传输对象的场景。

序列化和反序列化让你自己实现你会怎么做?

Java 默认的序列化虽然实现方便，但却存在安全漏洞、不跨语言以及性能差等缺陷。

- 无法跨语言： Java 序列化目前只适用基于 Java 语言实现的框架，其它语言大部分都没有使用 Java 的序列化框架，也没有实现 Java 序列化这套协议。因此，如果是两个基于不同语言编写的应用程序相互通信，则无法实现两个应用服务之间传输对象的序列化与反序列化。
- 容易被攻击：Java 序列化是不安全的，我们知道对象是通过在 ObjectInputStream 上调用 readObject() 方法进行反序列化的，这个方法其实是一个神奇的构造器，它可以将类路径上几乎所有实现了 Serializable 接口的对象都实例化。这也就意味着，在反序列化字节流的过程中，该方法可以执行任意类型的代码，这是非常危险的。
- 序列化后的流太大：序列化后的二进制流大小能体现序列化的性能。序列化后的二进制数组越大，占用的存储空间就越多，存储硬件的成本就越高。如果我们是进行网络传输，则占用的带宽就更多，这时就会影响到系统的吞吐量。

我会考虑用主流序列化框架，比如 FastJson、Protobuf 来替代 Java 序列化。

如果追求性能的话，Protobuf 序列化框架会比较合适，Protobuf 的这种数据存储格式，不仅压缩存储数据的效果好， 在编码和解码的性能方面也很高效。Protobuf 的编码和解码过程结合.proto 文件格式，加上 Protocol Buffer 独特的编码格式，只需要简单的数据运算以及位移等操作就可以完成编码与解码。可以说 Protobuf 的整体性能非常优秀。

将对象转为二进制字节流具体怎么实现?

其实，像序列化和反序列化，无论这些可逆操作是什么机制，都会有对应的**处理和解析协议**，例如加密和解密，TCP 的粘包和拆包，序列化机制是通过序列化协议来进行处理的，和 class 文件类

似，它其实是定义了序列化后的字节流格式，然后对此格式进行操作，生成符合格式的字节流或者将字节流解析成对象。

在 Java 中通过序列化对象流来完成序列化和反序列化：

- ObjectOutputStream：通过 writeObject() 方法做序列化操作。
- ObjectInputStrean：通过 readObject()方法做反序列化操作。

只有实现了 Serializable 或 Externalizable 接口的类的对象才能被序列化，否则抛出异常！

实现对象序列化：

- 让类实现 Serializable 接口：

```
import java.io.Serializable;


public class MyClass implements Serializable {
    // class code
}
```

- 创建输出流并写入对象：

```
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;


MyClass obj = new MyClass();
try {
    FileOutputStream fileOut = new FileOutputStream("object.ser");
    ObjectOutputStream out = new ObjectOutputStream(fileOut);
    out.writeObject(obj);
    out.close();
    fileOut.close();
} catch (IOException e) {
    e.printStackTrace();
}
```

实现对象反序列化：

- 创建输入流并读取对象：

```
import java.io.FileInputStream;
import java.io.ObjectInputStream;

MyClass newObj = null;
try {
    FileInputStream fileIn = new FileInputStream("object.ser");
    ObjectInputStream in = new ObjectInputStream(fileIn);
    newObj = (MyClass) in.readObject();
    in.close();
    fileIn.close();
} catch (IOException | ClassNotFoundException e) {
    e.printStackTrace();
}
```

通过以上步骤，对象 obj 会被序列化并写入到文件"object.ser"中，然后通过反序列化操作，从文件中读取字节流并恢复为对象 newObj。这种方式可以方便地将对象转换为字节流用于持久化存储、网络传输等操作。需要注意的是，要确保类实现了 Serializable 接口，并且所有成员变量都是 Serializable 的才能被正确序列化。

设计模式

volatile 和 sychronized 如何实现单例模式

```
public class SingleTon {

    // volatile 关键字修饰变量 防止指令重排序
    private static volatile SingleTon instance = null;
    private SingleTon(){}

    public static  SingleTon getInstance(){
        if(instance == null){
            //同步代码块 只有在第一次获取对象的时候会执行到 ，第二次及以后访问时 instance 变量均非null 故不会往下执行了 直接返回啦
            synchronized(SingleTon.class){
                if(instance == null){
                    instance = new SingleTon();
```

```
                }
            }
        }
        return instance;
    }
}
```

正确的双重检查锁定模式需要需要使用 volatile。volatile 主要包含两个功能。

- 保证可见性。使用 volatile 定义的变量，将会保证对所有线程的可见性。
- 禁止指令重排序优化。

由于 volatile 禁止对象创建时指令之间重排序，所以其他线程不会访问到一个未初始化的对象，从而保证安全性。

代理模式和适配器模式有什么区别?

- **目的不同**：代理模式主要关注控制对对象的访问，而适配器模式则用于接口转换，使不兼容的类能够一起工作。
- **结构不同**：代理模式一般包含抽象主题、真实主题和代理三个角色，适配器模式包含目标接口、适配器和被适配者三个角色。
- **应用场景不同**：代理模式常用于添加额外功能或控制对对象的访问，适配器模式常用于让不兼容的接口协同工作。

责任链模式使用场景是什么?

责任链模式的使用场景核心很明确，就是一个请求需要多个独立的处理逻辑来承接，同时不想让请求发起方和所有处理者产生强关联，还得让处理流程能灵活调整，简单说就是谁能处理就谁来接手，整个处理顺序和参与节点能按需改动。

比如实际开发里最常遇到的接口请求校验，用户调用我们的接口时，可能得先检查登录状态，再验证 token 是否有效，接着确认接口访问权限，最后还要限制请求频率，这些校验逻辑各自独立，而且不同接口需要的校验步骤不一样，比如登录接口只需要验证验证码，查询用户信息的接口得同时过登录和权限校验。要是不用责任链，就得在每个接口里写一堆 if-else 把这些校验串起来，后续想改某个校验规则，所有相关接口都得动，维护起来特别麻烦。

这时候用责任链就很合适，我们可以把每个校验逻辑封装成一个处理节点，先定义一个抽象的处理者类：

```
// 抽象处理者
abstract class Handler {
    protected Handler next;
    // 设置下一个处理节点
    public void setNext(Handler next) {
        this.next = next;
    }
    // 抽象处理方法
    public abstract boolean handle(Request request);
}
```

然后每个校验逻辑都继承这个类，比如登录校验：

```
// 登录态校验节点
class LoginHandler extends Handler {
    @Override
    public boolean handle(Request request) {
        if (request.isLogin()) {
            System.out.println("登录态校验通过，交给下一个节点");
            // 校验通过，交给下一个处理者
            return next != null ? next.handle(request) : true;
        } else {
            System.out.println("未登录，直接返回失败");
            // 校验不通过，终止链路
            returnfalse;
        }
    }
}
```

再写个权限校验节点、频率限制节点，最后在使用的时候，根据接口需求动态组装链路：

```
// 组装链路：登录校验 -> 权限校验 -> 频率限制
Handler loginHandler = new LoginHandler();
Handler authHandler = new AuthHandler();
Handler rateLimitHandler = new RateLimitHandler();
loginHandler.setNext(authHandler);
```

```
authHandler.setNext(rateLimitHandler);


// 发起请求
Request request = new Request(true, "admin", 1); // 已登录、管理员权限、第 1 次请求
boolean result = loginHandler.handle(request);
```

这样一来，请求发起方只需要调用第一个节点，根本不用关心后面有多少校验步骤，也不用知道具体是哪个节点在处理；如果某个接口不需要频率限制，直接去掉 rateLimitHandler 的组装就行，不用修改任何校验节点的代码，流程调整起来特别灵活。

介绍一下策略模式和责任链模式，分别用在哪些场景?

这两个设计模式都属于行为型模式，而且在实际开发中，它们往往都是为了解决同一个痛点：**如何消除代码中复杂的 `**if-else**` 或 `**switch-case**` 逻辑，从而让系统更易于扩展。**

> 策略模式

策略模式定义一系列算法，把它们一个个封装起来，并且使它们可相互替换。从而让算法可独立于使用它的客户而变化。

假设你正在开发一个电商网站的支付模块。用户可以选择支付宝、微信支付或者银行卡支付。每种支付方式的底层逻辑（比如调用的 API、加密方式、参数构造）都完全不同，但对于调用支付功能的上层业务逻辑来说，它只需要调用一个统一的 “支付” 方法即可，无需关心具体是哪种支付。

这就是策略模式的用武之地。我们把每种支付方式（支付宝、微信）都封装成一个独立的 “策略” 类，它们都遵循一个共同的接口。上层业务代码通过这个共同的接口来调用支付功能，并可以在运行时根据用户的选择，动态地切换不同的策略。

所以，策略模式**适用场景如下：**

- 当一个系统需要动态地在几种算法中选择一种时。
- 当一个对象有很多行为，不想使用大量的 `if-else` 或 `switch-case` 语句来选择时。
- 当你希望算法的变化不会影响到使用它的客户端时。

> 责任链模式

责任链模式核心思想是为请求创建一个接收者对象的链。对请求的发送者和接收者进行解耦，让多个对象都有机会处理这个请求。请求沿着链传递，直到链上的一个对象处理它为止。

想象一下公司里的请假审批流程。一个员工要请假，他首先会把请假条交给自己的直属经理。如果请假天数很短（比如 1 天），经理自己就能批准。如果天数较长（比如 5 天），经理没有权限，就需要把请假条上报给部门总监。总监也有自己的审批权限上限，如果超过了，就需要继续上报给总经理。

在这个过程中，请假条（请求）就在一条由经理 -> 总监 -> 总经理（接收者链）上传递，直到某个角色（接收者）能够处理它。发送请假条的员工（客户端）并不知道最终是谁批准了他的假，他只需要把请求提交给链的第一个节点（经理）即可。

所以，责任链模式适用场景总结：

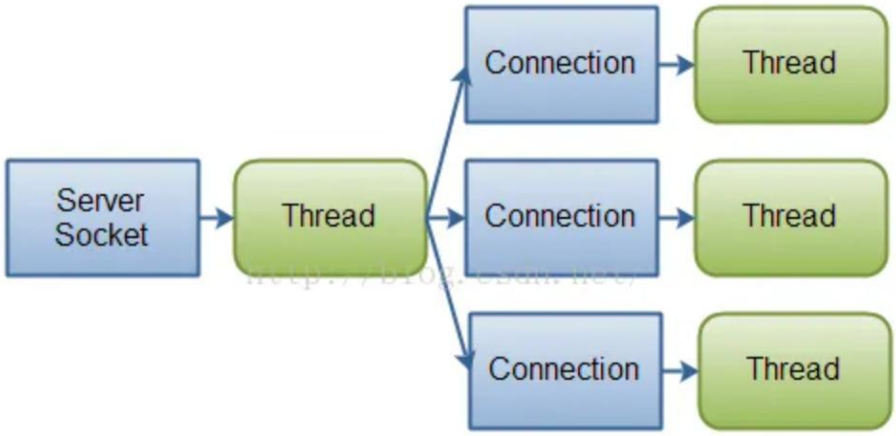
- 当有多个对象可以处理同一个请求，但具体由哪个对象处理需要在运行时决定时。
- 当你希望请求的发送者和接收者解耦，不直接通信时。
- 当你需要动态地指定和修改处理一个请求的对象集合时。

I/O

Java 怎么实现网络 IO 高并发编程?

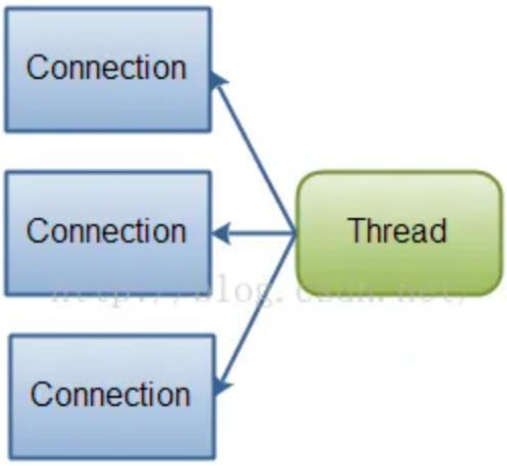
可以用 Java NIO ，是一种同步非阻塞的 I/O 模型，也是 I/O 多路复用的基础。

传统的 BIO 里面 socket.read()，如果 TCP RecvBuffer 里没有数据，函数会一直阻塞，直到收到数据，返回读到的数据， 如果使用 BIO 要想要并发处理多个客户端的 i/o，那么会使用多线程模式，一个线程专门处理一个客户端 io，这种模式随着客户端越来越多，所需要创建的线程也越来越多，会急剧消耗系统的性能。



Java IO: A classic IO server design - one connection handled by one thread.

NIO 是基于 I/O 多路复用实现的，它可以只用一个线程处理多个客户端 I/O，如果你需要同时管理成千上万的连接，但是每个连接只发送少量数据，例如一个聊天服务器，用 NIO 实现会更好一些。



Java NIO: A single thread managing multiple connections.

BIO、NIO、AIO 区别是什么?

- BIO（blocking IO）：就是传统的 java.io 包，它是基于流模型实现的，交互的方式是同步、阻塞方式，也就是说在读入输入流或者输出流时，在读写动作完成之前，线程会一直阻塞在那里，它们之间的调用是可靠的线性顺序。优点是代码比较简单、直观；缺点是 IO 的效率和扩展性很低，容易成为应用性能瓶颈。
- NIO（non-blocking IO）：Java 1.4 引入的 java.nio 包，提供了 Channel、Selector、Buffer 等新的抽象，可以构建多路复用的、同步非阻塞 IO 程序，同时提供了更接近操作系统底层高性能的数据操作方式。
- AIO（Asynchronous IO）：是 Java 1.7 之后引入的包，是 NIO 的升级版本，提供了异步非堵塞的 IO 操作方式，所以人们叫它 AIO（Asynchronous IO），异步 IO 是基于事件和回调机制实现的，也就是应用操作之后会直接返回，不会堵塞在那里，当后台处理完成，操作系统会通知相应的线程进行后续的操作。

NIO 是怎么实现的?

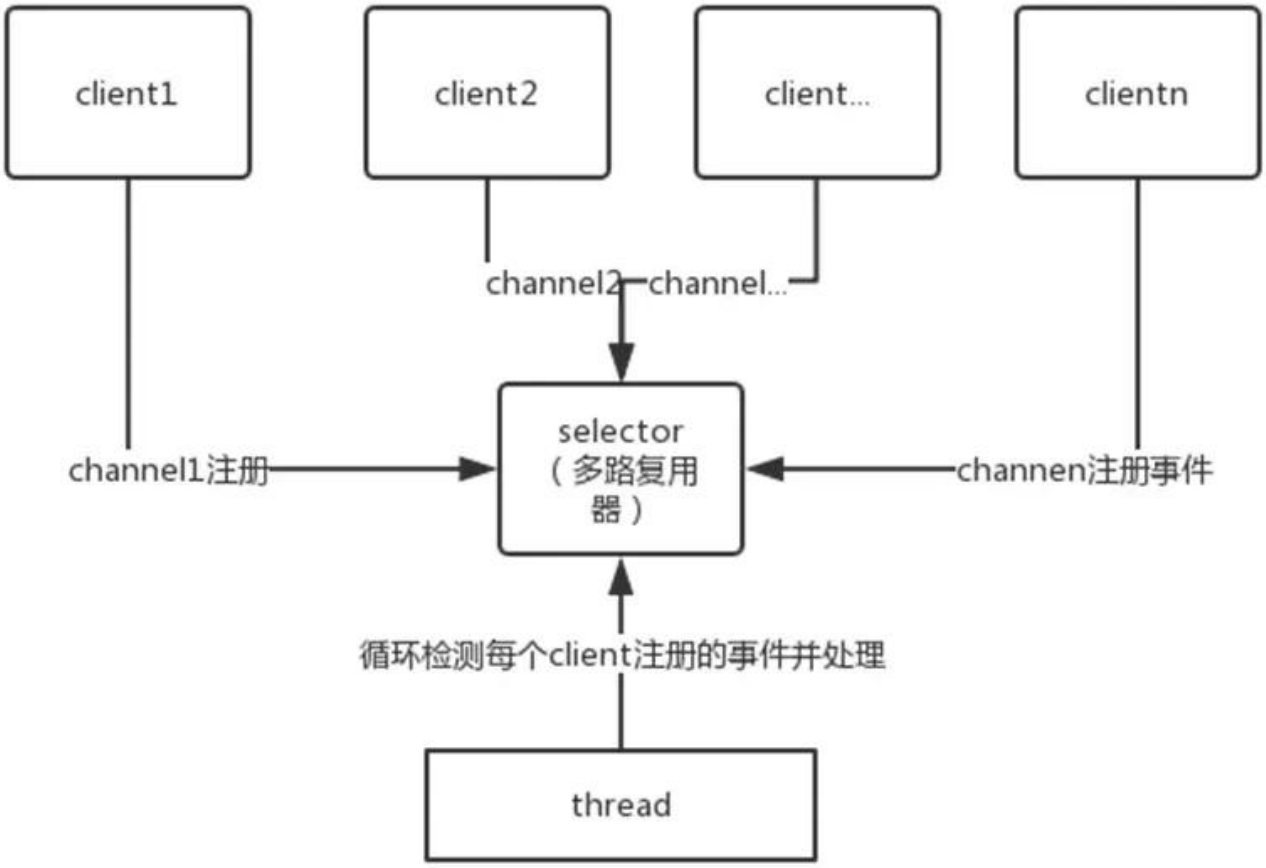
NIO 是一种同步非阻塞的 IO 模型，所以也可以叫 NON-BLOCKINGIO。同步是指线程不断轮询 IO 事件是否就绪，非阻塞是指线程在等待 IO 的时候，可以同时做其他任务。

同步的核心就 Selector（I/O 多路复用），Selector 代替了线程本身轮询 IO 事件，避免了阻塞同时减少了不必要的线程消耗；非阻塞的核心就是通道和缓冲区，当 IO 事件就绪时，可以通过写到缓冲区，保证 IO 的成功，而无需线程阻塞式地等待。

NIO 由一个专门的线程处理所有 IO 事件，并负责分发。事件驱动机制，事件到来的时候触发操作，不需要阻塞的监视事件。线程之间通过 wait,notify 通信，减少线程切换。

NIO 主要有三大核心部分：Channel(通道)，Buffer(缓冲区)，Selector。传统 IO 基于字节流和字符流进行操作，而 NIO 基于 Channel 和 Buffer(缓冲区)进行操作，数据总是从通道读取到缓冲区中，或者从缓冲区写入到通道中。

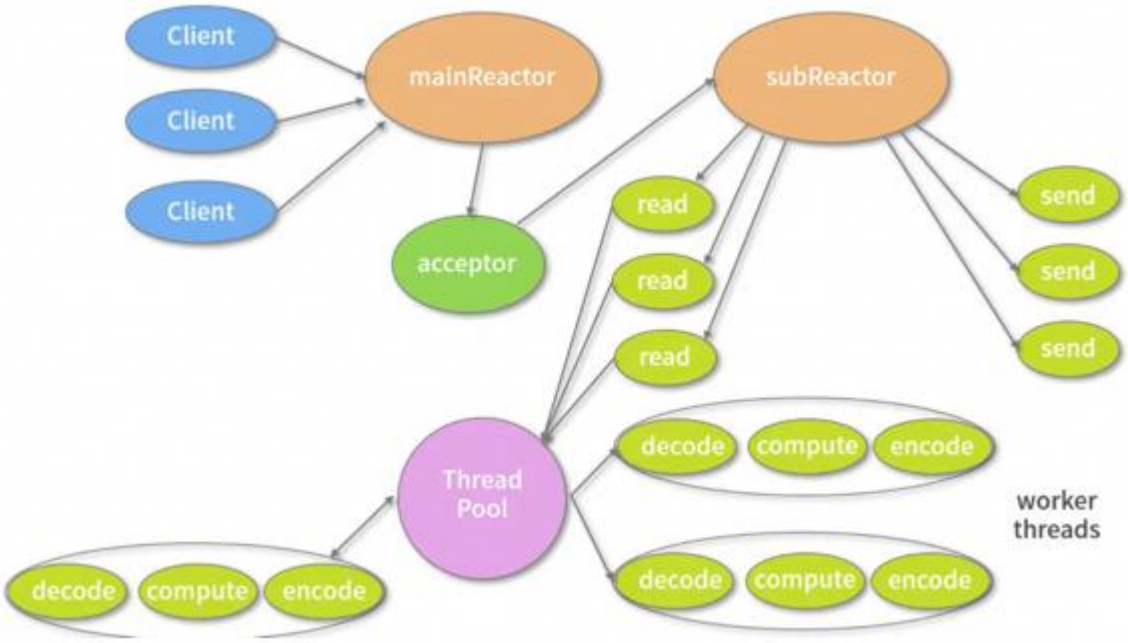
Selector(选择区)用于监听多个通道的事件（比如：连接打开，数据到达）。因此，单个线程可以监听多个数据通道。



你知道有哪个框架用到 NIO 了吗?

Netty。

Netty 的 I/O 模型是基于非阻塞 I/O 实现的，底层依赖的是 NIO 框架的多路复用器 Selector。采用 epoll 模式后，只需要一个线程负责 Selector 的轮询。当有数据处于就绪状态后，需要一个事件分发器（Event Dispather），它负责将读写事件分发给对应的读写事件处理器（Event Handler）。事件分发器有两种设计模式：Reactor 和 Proactor，Reactor 采用同步 I/O，Proactor 采用异步 I/O。



Reactor 实现相对简单，适合处理耗时短的场景，对于耗时长的 I/O 操作容易造成阻塞。Proactor 性能更高，但是实现逻辑非常复杂，适合图片或视频流分析服务器，目前主流的事件驱动模型还是依赖 select 或 epoll 来实现。

其他

有一个学生类，想按照分数排序，再按学号排序，应该怎么做?

可以使用 Comparable 接口来实现按照分数排序，再按照学号排序。首先在学生类中实现 Comparable 接口，并重写 compareTo 方法，然后在 compareTo 方法中实现按照分数排序和按照学号排序的逻辑。

```
public class Student implements Comparable<Student> {
    private int id;
    private int score;

    // 构造方法和其他属性、方法省略

    @Override
    public int compareTo(Student other) {
        if (this.score != other.score) {
            return Integer.compare(other.score, this.score); // 按照分数降序排序
        } else {
```

```
            return Integer.compare(this.id, other.id); // 如果分数相同，则按照学号升序排序
        }
    }
}
```

然后在需要对学生列表进行排序的地方，使用 Collections.sort()方法对学生列表进行排序即可：

```
List<Student> students = new ArrayList<>();
// 添加学生对象到列表中
Collections.sort(students);
```

Native 方法解释一下

在 Java 中，native 方法是一种特殊类型的方法，它允许 Java 代码调用外部的本地代码，即用 C、C++或其他语言编写的代码。native 关键字是 Java 语言中的一种声明，用于标记一个方法的实现将在外部定义。

在 Java 类中，native 方法看起来与其他方法相似，只是其方法体由 native 关键字代替，没有实际的实现代码。例如：

```
public class NativeExample {
    public native void nativeMethod();
}
```

要实现 native 方法，你需要完成以下步骤：

1. **生成 JNI 头文件**：使用 javah 工具从你的 Java 类生成 C/C++的头文件，这个头文件包含了所有 native 方法的原型。
2. **编写本地代码**：使用 C/C++编写本地方法的实现，并确保方法签名与生成的头文件中的原型匹配。
3. **编译本地代码**：将 C/C++代码编译成动态链接库（DLL，在 Windows 上），共享库（SO，在 Linux 上）
4. **加载本地库**：在 Java 程序中，使用 System.loadLibrary()方法来加载你编译好的本地库，这样 JVM 就能找到并调用 native 方法的实现了。

Java 进程是怎么跟操作系统交互的?

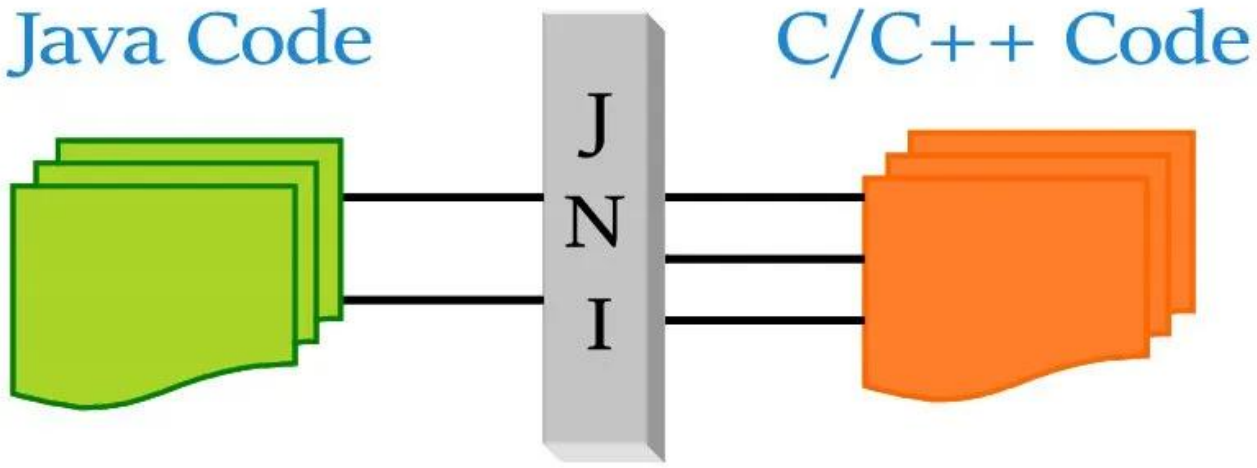
Java 程序虽然是运行在 JVM 上的，但 JVM 本身就是一个操作系统进程，所以 Java 跟操作系统的交互，本质上是通过 JVM 这个中间层来完成的。

最基础的交互就是内存管理。JVM 启动时会通过系统调用向操作系统申请内存，比如在 Linux 上用 mmap、brk 这些系统调用来分配堆内存、栈内存。虽然我们在 Java 代码里写 new 对象，但底层还是要跟操作系统打交道获取真实的物理内存。

线程管理也是重要的交互点。Java 里创建的线程对应操作系统的原生线程，在 Linux 上会调用 pthread_create 来创建线程。线程的调度、同步机制，比如 synchronized 底层用的互斥锁，最终都是由操作系统来管理的。

IO 操作是最频繁的交互场景。文件读写会调用 read、write 这些系统调用，网络编程底层也是调用 socket、bind、listen 这些系统调用。Java 的 NIO 用的是操作系统提供的高性能 IO 机制，Linux 上是 epoll，Windows 上是 IOCP。

JNI 是另一个重要通道。有些操作 Java 做不了或者性能不够，就通过 JNI 调用 C/C++代码，这些本地代码可以直接调用操作系统 API。JVM 本身也大量使用 JNI 来实现底层功能。



还有信号处理，比如你用 kill 命令给 Java 进程发 SIGTERM 信号，JVM 会捕获这个信号然后执行关闭钩子。进程间通信、获取系统资源信息这些，底层也都是通过系统调用完成的。

从用户态和内核态的角度说，Java 代码运行在用户态，但 IO、网络这些操作需要切换到内核态，这个切换是有开销的。JVM 会做很多优化，比如用缓冲区减少系统调用次数。虽然 JVM 屏蔽了底层差异让 Java 代码跨平台，但在每个平台上 JVM 都要适配该平台的系统调用和 API。
